

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина: Компьютерные системы и сети

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему

ВЕБ-ПРИЛОЖЕНИЕ ОБЛАЧНОГО ФАЙЛОВОГО ХРАНИЛИЩА
БГУИР КП 6-05-0612-01 021 ПЗ

Студент

Ковалевич К.А.

Руководитель

Болтак С.В.

Минск 2026

СОДЕРЖАНИЕ

Введение	5
1 Анализ предметной области	7
1.1 Сравнительный анализ существующих решений	7
1.2 Постановка задачи	11
2 Проектирование программного средства	13
2.1 Архитектура системы и структура сетевого взаимодействия	13
2.2 Проектирование взаимодействия с внешним API	16
2.3 Проектирование пользовательского интерфейса	16
3 Разработка программного средства	21
3.1 Реализация серверной части веб-приложения	21
3.2 Реализация механизмов получения и обработки данных	24
3.3 Реализация системы авторизации и персонализации пользователей....	24
3.4 Реализация клиентской части веб-приложения	28
4 Тестирование программного средства	30
5 Руководство пользователя	38
Заключение	46
Список используемых источников	46
Приложение А (обязательное) Листинг кода с комментариями	49
Приложение Б (обязательное) Схема алгоритма работы системы	65

ВВЕДЕНИЕ

Курсовой проект посвящён разработке веб-приложения облачного файлового хранилища. Такое программное средство предназначено для размещения пользовательских файлов на сервере и управления ими через браузер. В рамках проекта реализуется клиент-серверное приложение на языке Python, в котором обмен данными между пользователем и сервером выполняется с использованием протокола HTTP, метаданные сохраняются в JSON-файлах, а сами загруженные файлы размещаются в файловой системе сервера. Эти требования соответствуют листу задания по курсовому проектированию.

Необходимость разработки подобного приложения связана с тем, что пользователю важно иметь удобный доступ к своим данным без привязки к конкретному устройству. При локальном хранении файлы доступны только на одном компьютере, а при использовании веб-хранилища пользователь может работать с ними через обычный браузер. Такой подход удобен для учебных материалов, документов, изображений, архивов и других типов файлов, которые требуется быстро загрузить, найти, скачать или удалить.

Разрабатываемое программное средство можно рассматривать как учебный аналог облачных сервисов хранения данных. В отличие от крупных коммерческих платформ, таких как Google Drive, Яндекс Диск или Dropbox, данное приложение не ориентировано на массовую эксплуатацию и сложную инфраструктуру. Основная задача проекта заключается в демонстрации принципов построения веб-системы: обработки запросов клиента, авторизации пользователей, разграничения доступа, работы с файлами, хранения метаданных и организации серверной логики.

Целью курсового проекта является разработка веб-приложения облачного файлового хранилища, позволяющего зарегистрированным пользователям загружать, хранить и управлять файлами через веб-интерфейс.

Для достижения поставленной цели необходимо выполнить анализ предметной области и рассмотреть особенности существующих файловых хранилищ; определить требования к разрабатываемому приложению; спроектировать структуру клиент-серверного взаимодействия; разработать серверную часть на языке Python с использованием фреймворка Flask; реализовать регистрацию, авторизацию и роли пользователей; организовать хранение файлов в каталогах сервера, а сведений о них – в JSON-файлах; создать пользовательский интерфейс для загрузки, скачивания, поиска, сортировки, переименования, перемещения и удаления файлов; реализовать корзину, избранное, квоты хранения и административную панель; провести тестирование основных функций приложения.

Объектом разработки является процесс хранения и управления пользовательскими файлами в веб-среде. Предметом разработки является веб-приложение, реализующее функции облачного файлового хранилища средствами языка Python и фреймворка Flask.

Практическая значимость проекта состоит в том, что разработанная система позволяет изучить основные механизмы работы современных веб-приложений. В ходе реализации рассматриваются такие важные аспекты, как маршрутизация HTTP-запросов, работа с пользовательскими сессиями, проверка прав доступа, обработка загружаемых файлов, ведение журнала действий, работа с JSON-репозиториями и структурирование данных в файловой системе сервера.

Разрабатываемое приложение имеет модульную структуру. В нём выделены модели данных, репозитории для работы с JSON-файлами, сервисы бизнес-логики и маршруты Flask, отвечающие за обработку действий пользователя. Такой подход упрощает сопровождение программы, делает код более понятным и соответствует принципам объектно-ориентированного программирования.

Таким образом, результатом выполнения курсового проекта является работоспособное веб-приложение CloudBox, обеспечивающее пользователю возможность безопасно хранить файлы на сервере, управлять ими через браузер и получать доступ к основным функциям облачного файлового хранилища в рамках локально развёрнутой системы.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Сравнительный анализ существующих решений

Перед разработкой собственного веб-приложения облачного файлового хранилища необходимо рассмотреть уже существующие решения, которые используются для хранения, передачи и организации пользовательских файлов. Анализ аналогов позволяет определить основные функции, которые являются обязательными для подобных систем, а также выявить недостатки, которые можно учесть при проектировании собственного программного средства.

В качестве аналогов были выбраны три популярных облачных файловых сервиса: Google Drive, Яндекс Диск и Dropbox. Данные платформы предоставляют пользователю возможность хранить файлы на удалённом сервере, работать с ними через браузер, синхронизировать данные между устройствами и организовывать файлы по папкам. Несмотря на схожее назначение, каждый сервис имеет собственные особенности интерфейса, функциональности и организации работы с данными.

Первым рассмотренным решением является Google Drive. Это облачное файловое хранилище, тесно связанное с экосистемой Google. Пользователь может загружать документы, изображения, архивы и другие файлы, создавать папки, выполнять поиск, открывать файлы в браузере и предоставлять доступ другим пользователям. Одним из преимуществ Google Drive является удобная интеграция с Google Документами, Google Таблицами и другими онлайн-сервисами. Благодаря этому пользователь может не только хранить данные, но и редактировать часть документов прямо в браузере.

На рисунке 1.1 представлен интерфейс облачного сервиса Google Drive.

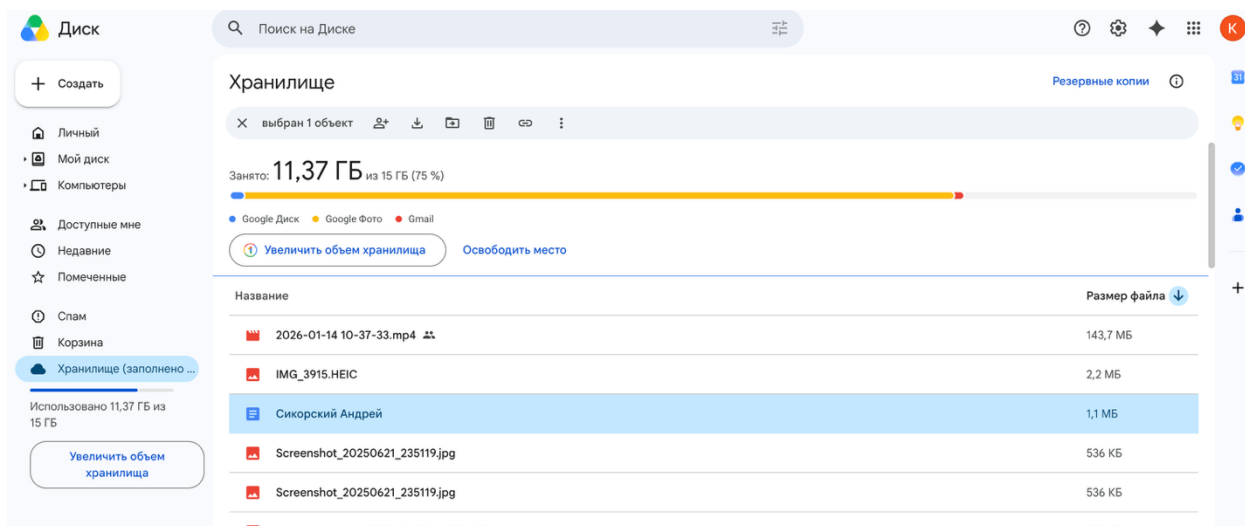


Рисунок 1.1 – Интерфейс облачного файлового хранилища Google Drive

Вторым аналогом является Яндекс Диск. Данный сервис также предназначен для хранения пользовательских файлов в облаке и предоставляет доступ к ним через браузер или специальное приложение. Пользователь может загружать файлы, создавать папки, просматривать изображения, делиться ссылками и синхронизировать данные между устройствами. Интерфейс Яндекс Диска ориентирован на массового пользователя и содержит привычную структуру файлового менеджера: папки, список файлов, корзину, поиск и панель действий.

Преимуществом Яндекс Диска является понятный интерфейс и наличие функций, которые удобно использовать в повседневной работе: корзина, сортировка файлов, быстрый просмотр изображений и возможность делиться файлами по ссылке. Однако сервис привязан к аккаунту Яндекс и является частью крупной облачной платформы. Для работы требуется подключение к внешнему серверу, а управление внутренней логикой хранения данных пользователю недоступно. В рамках курсового проекта важно не только получить готовую возможность хранения файлов, но и самостоятельно реализовать серверную обработку запросов, структуру хранения метаданных и работу с файловой системой.

На рисунке 1.2 представлен интерфейс облачного сервиса Яндекс Диск.

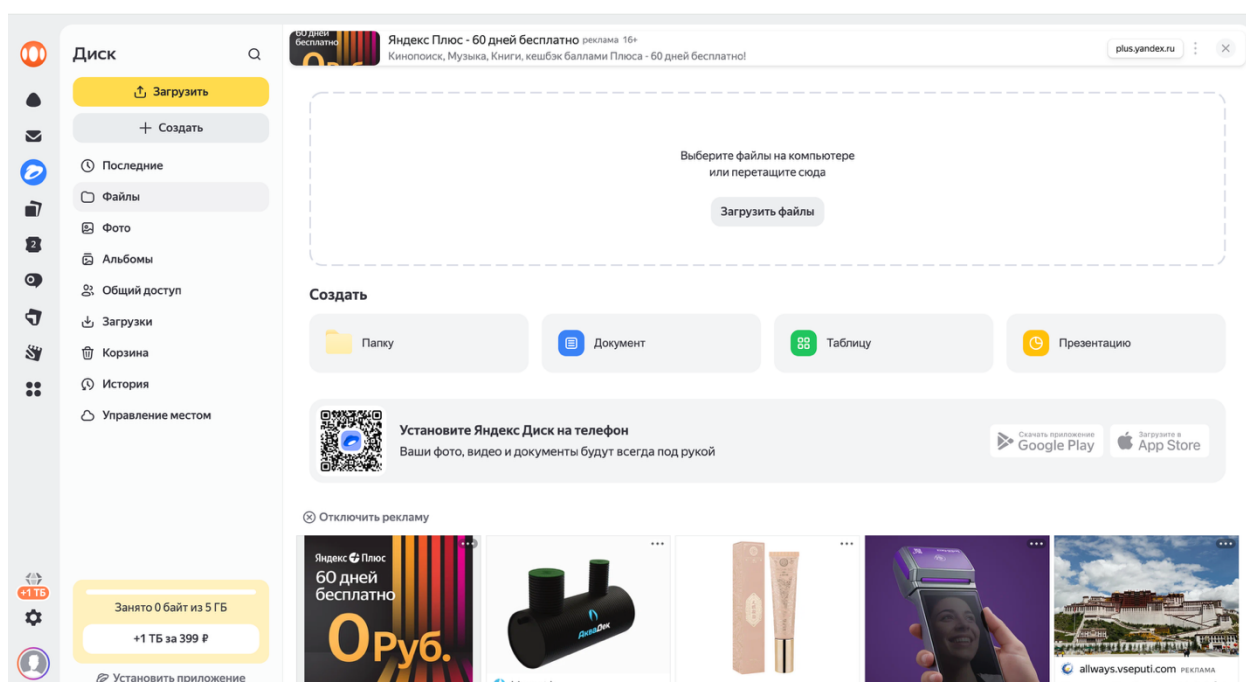


Рисунок 1.2 – Интерфейс облачного файлового хранилища Яндекс Диск

Третьим рассмотренным решением является Dropbox. Это один из известных сервисов для хранения и синхронизации файлов между устройствами. Dropbox ориентирован на удобную работу с документами, папками и общими ресурсами. Пользователь может загружать файлы, создавать каталоги, открывать общий доступ, отслеживать изменения и использовать синхронизацию с локальной папкой на компьютере. Сервис часто применяется для совместной работы и обмена файлами между пользователями.

Главным преимуществом Dropbox является простота интерфейса и удобная синхронизация данных. Однако, как и другие коммерческие облачные платформы, Dropbox зависит от внешней инфраструктуры и не позволяет пользователю управлять внутренним механизмом хранения. Также значительная часть расширенных возможностей связана с тарифными планами и корпоративными функциями. Для проектирования собственного решения целесообразно оставить только основные действия с файлами и реализовать их в более простой и понятной форме.

На рисунке 1.3 представлен интерфейс облачного сервиса Dropbox.

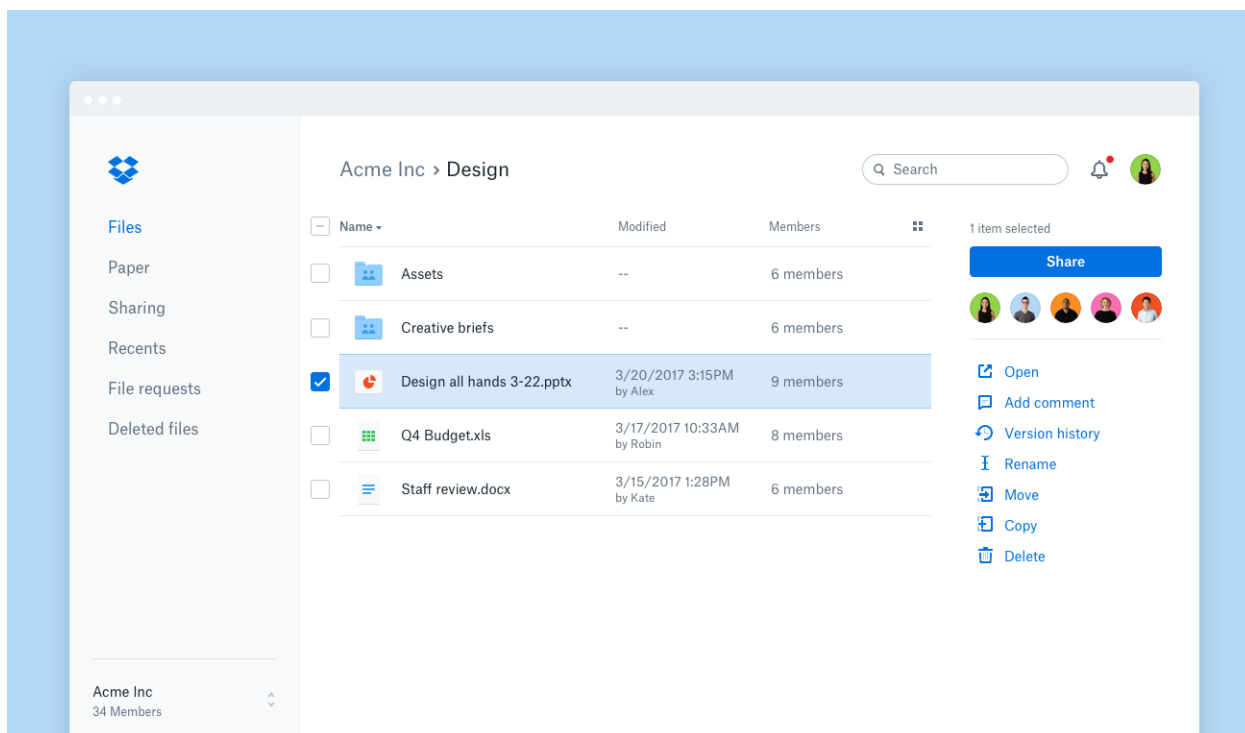


Рисунок 1.3 – Интерфейс облачного файлового хранилища Dropbox

Проведённый анализ показывает, что все рассмотренные сервисы имеют общий набор базовых возможностей: загрузку файлов, создание папок, скачивание, удаление, поиск, сортировку и работу с корзиной. Эти функции являются основой любого облачного файлового хранилища, поэтому они должны быть предусмотрены и в разрабатываемом приложении CloudBox. При этом крупные облачные сервисы обладают большим количеством дополнительных возможностей, которые усложняют интерфейс и требуют развитой серверной инфраструктуры.

Разрабатываемое приложение CloudBox отличается от рассмотренных аналогов тем, что создаётся как учебная локальная система. Оно не требует подключения к сторонним облачным платформам и не использует внешнюю базу данных. Метаданные пользователей, файлов и журнала действий сохраняются в JSON-файлах, а сами загруженные файлы размещаются в файловой системе сервера. Такой подход делает программу более простой для развёртывания, изучения и сопровождения.

В CloudBox предусмотрены основные функции, характерные для облачных хранилищ: регистрация и авторизация пользователей, загрузка файлов, просмотр содержимого личного хранилища, создание папок, скачивание, переименование, перемещение, удаление файлов, восстановление из корзины, поиск, сортировка и отметка файлов как избранных. Дополнительно реализуется административная панель, позволяющая просматривать пользователей, статистику системы и журнал действий.

Таким образом, сравнительный анализ существующих решений показал, что разработка собственного веб-приложения облачного файлового хранилища является обоснованной. В отличие от крупных коммерческих сервисов, CloudBox ориентирован на демонстрацию основных принципов клиент-серверного взаимодействия, обработки HTTP-запросов, хранения файлов на сервере и управления метаданными. Это позволяет создать законченное программное средство, соответствующее теме курсового проекта и требованиям учебной разработки.

1.2 Постановка задачи

В рамках курсового проекта необходимо разработать веб-приложение облачного файлового хранилища, предназначенное для размещения пользовательских файлов на сервере и управления ими через браузер. Программное средство должно обеспечивать работу с файлами в личном кабинете пользователя, поддерживать регистрацию и авторизацию, а также предоставлять базовые административные функции для контроля состояния системы.

Основной задачей разрабатываемого приложения является создание удобного веб-интерфейса, через который пользователь сможет выполнять основные операции с файлами без прямого обращения к файловой системе сервера. Пользователь должен иметь возможность загружать файлы в своё личное хранилище, просматривать список сохранённых объектов, скачивать необходимые файлы, переименовывать их, перемещать между папками, удалять в корзину и при необходимости восстанавливать. Также необходимо предусмотреть функции поиска, сортировки и отметки файлов как избранных.

Разрабатываемая система построена по клиент-серверной архитектуре. Клиентская часть представляет собой веб-страницы, открываемые пользователем в браузере. Серверная часть должна быть реализована на языке Python с использованием веб-фреймворка Flask. Сервер принимает HTTP-запросы от клиента, выполняет проверку прав доступа, обрабатывает действия пользователя и возвращает результат в виде HTML-страниц. Такой подход соответствует требованиям задания, где указано, что приложение должно работать через браузер, использовать HTTP/HTTPS, сохранять метаданные в JSON-файлы, а сами файлы размещать в файловой системе сервера.

Для хранения данных в проекте необходимо использовать файловый подход. Сведения о пользователях, файлах и действиях системы должны храниться в отдельных JSON-файлах. Такой вариант хранения позволяет отказаться от использования внешней базы данных и упростить развёртывание приложения. При этом сами загруженные пользователями файлы должны размещаться в специальном каталоге серверной файловой системы. Для

каждого пользователя должно быть создано отдельное пространство хранения, что позволит изолировать данные разных пользователей друг от друга.

В приложении необходимо реализовать две роли доступа: обычный пользователь и администратор. Обычный пользователь должен работать только со своими файлами и папками. Администратор должен иметь возможность просматривать список зарегистрированных пользователей, получать общую статистику по системе, изучать журнал действий, а также блокировать, разблокировать и удалять пользователей. Разделение ролей требуется для обеспечения базового контроля доступа и повышения безопасности системы.

Особое внимание при разработке следует уделить безопасности пользовательских данных. Пароли не должны храниться в открытом виде, поэтому необходимо использовать хеширование. Также приложение должно проверять, авторизован ли пользователь перед выполнением защищённых действий. Пользователь не должен иметь доступ к файлам других пользователей, даже если он попытается изменить адрес страницы или идентификатор файла вручную.

Необходимо предусмотреть обработку ошибок, которые могут возникнуть при работе с приложением. К таким ситуациям относятся попытка загрузить файл слишком большого размера, обращение к несуществующему файлу, ввод неверных данных при входе, попытка регистрации пользователя с уже существующим email, удаление отсутствующего объекта или переход на несуществующую страницу. В подобных случаях система должна не завершаться аварийно, а выводить пользователю понятное сообщение.

В ходе разработки требуется реализовать модульную структуру программного средства. Модели должны описывать основные сущности предметной области, такие как пользователь, файл и запись журнала. Репозитории должны отвечать за чтение и запись JSON-файлов. Сервисный слой должен содержать основную бизнес-логику: регистрацию, авторизацию, загрузку файлов, удаление, восстановление, поиск, сортировку и подсчёт статистики. Маршруты Flask должны принимать запросы от пользователя и связывать веб-интерфейс с внутренней логикой приложения.

Результатом выполнения поставленной задачи должно стать работоспособное веб-приложение CloudBox, позволяющее пользователям хранить файлы на сервере и управлять ими через браузер. Приложение должно демонстрировать принципы клиент-серверного взаимодействия, обработку HTTP-запросов, работу с пользовательскими сессиями, разграничение прав доступа, хранение метаданных в JSON-формате и организацию файлового пространства на сервере.

2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

2.1 Архитектура системы и структура сетевого взаимодействия

При проектировании веб-приложения CloudBox была выбрана клиент-серверная архитектура. Такой подход позволяет разделить систему на две основные части: клиентскую, с которой взаимодействует пользователь, и серверную, выполняющую обработку запросов, проверку прав доступа, работу с файлами и сохранение данных. Клиентская часть открывается в браузере пользователя, а серверная часть реализуется на языке Python с использованием фреймворка Flask.

Клиентская часть приложения представляет собой набор HTML-страниц, формируемых сервером и отображаемых в браузере. Через веб-интерфейс пользователь может выполнять регистрацию, вход в систему, загрузку файлов, просмотр содержимого личного хранилища, создание папок, поиск, сортировку, скачивание, переименование, перемещение и удаление файлов. Также в интерфейсе предусмотрены отдельные страницы для работы с избранными файлами, корзиной и административной панелью.

Серверная часть является центральным компонентом системы. Она принимает HTTP-запросы от браузера, определяет требуемое действие, обращается к сервисному слою, выполняет операции с JSON-файлами и файловой системой, после чего возвращает пользователю сформированную HTML-страницу или файл для скачивания. В качестве основы серверной части используется Flask, так как данный фреймворк позволяет быстро создавать веб-приложения, описывать маршруты обработки запросов и удобно разделять проект на отдельные модули.

Сетевое взаимодействие между клиентом и сервером выполняется по протоколу HTTP. Пользователь отправляет запросы через браузер, например при переходе на страницу, отправке формы регистрации, загрузке файла или нажатии кнопки удаления. Сервер Flask получает запрос, обрабатывает его и формирует ответ. Для получения страниц и данных используются GET-запросы, а для действий, изменяющих состояние системы, применяются POST-запросы. К таким действиям относятся регистрация, авторизация, загрузка файла, создание папки, переименование, перемещение, удаление и восстановление файлов.

Архитектура приложения построена по модульному принципу. В проекте выделены маршруты, сервисы, репозитории и модели. Маршруты отвечают за приём HTTP-запросов и передачу данных между веб-интерфейсом и внутренней логикой приложения. Сервисный слой содержит основную бизнес-логику: регистрацию пользователей, проверку паролей, работу с

файлами, подсчёт занятого места, поиск, сортировку, работу с корзиной и обработку действий администратора. Репозитории предназначены для чтения и записи данных в JSON-файлы. Модели описывают основные сущности предметной области: пользователя, файл и запись журнала.

Для хранения данных в системе используется файловый подход. Метаданные пользователей сохраняются в файле *users.json*, сведения о загруженных файлах – в файле *files.json*, а журнал действий – в файле *logs.json*. Сами пользовательские файлы не помещаются в JSON-файлы, а сохраняются в файловой системе сервера в отдельном каталоге. Для каждого пользователя создаётся собственная директория вида *storage/users/user_<id>/*, что позволяет логически отделить файлы разных пользователей друг от друга.

Такой способ хранения соответствует назначению учебного проекта, так как не требует установки и настройки отдельной системы управления базами данных. JSON-файлы позволяют сохранить структурированную информацию в понятном виде, а файловая система сервера используется для хранения реального содержимого пользовательских файлов. При этом серверная логика связывает физический файл с его метаданными: именем, размером, расширением, датой загрузки, владельцем, папкой размещения и дополнительными признаками, например нахождением в корзине или в избранном.

В системе предусмотрено разграничение прав доступа. Неавторизованный пользователь может перейти только на страницы регистрации и входа. После успешной авторизации пользователь получает доступ к личному кабинету и может работать только со своими файлами. Администратор имеет расширенные права и может просматривать список пользователей, статистику системы и журнал действий. Для защиты закрытых страниц используются специальные проверки авторизации и роли пользователя.

Безопасность работы приложения обеспечивается несколькими механизмами. Пароли пользователей сохраняются не в открытом виде, а в виде кеша. При попытке входа введённый пароль сравнивается с сохранённым кешем. Кроме того, перед выполнением операций с файлами сервер проверяет, принадлежит ли выбранный файл текущему пользователю. Это предотвращает доступ одного пользователя к данным другого пользователя через изменение адреса страницы или идентификатора файла.

Общая схема взаимодействия компонентов системы выглядит следующим образом. Пользователь открывает веб-приложение в браузере и отправляет HTTP-запрос. Flask-приложение принимает запрос и передаёт его соответствующему маршруту. Маршрут обращается к сервису, который выполняет необходимую операцию. Если требуется работа с данными, сервис

обращается к репозиторию, который читает или изменяет JSON-файл. Если требуется операция с файлом, сервис обращается к файловой системе сервера. После выполнения действия сервер формирует ответ и отправляет его обратно в браузер пользователя.

Таким образом, выбранная архитектура обеспечивает понятное разделение обязанностей между частями приложения. Клиентская часть отвечает за отображение интерфейса и отправку запросов, серверная часть – за обработку действий пользователя, сервисный слой – за бизнес-логику, репозитории – за работу с JSON-данными, а файловая система – за хранение пользовательских файлов. Такая структура делает приложение CloudBox удобным для разработки, тестирования и дальнейшего сопровождения.

2.2 Проектирование взаимодействия с внешним API

При проектировании веб-приложения CloudBox особое внимание было уделено организации хранения пользовательских данных. Так как разрабатываемая система является учебным облачным файловым хранилищем, было принято решение не использовать внешнюю систему управления базами данных. Вместо этого метаданные приложения сохраняются в JSON-файлах, а сами пользовательские файлы размещаются в файловой системе сервера. Такой подход соответствует заданию на курсовое проектирование, где указано, что метаданные должны храниться в JSON-файлах, а файлы – в выделенном каталоге серверной файловой системы.

В системе выделяются два вида хранимой информации: служебные данные и пользовательские файлы. К служебным данным относятся сведения об учётных записях пользователей, информация о загруженных файлах, а также журнал действий. Эти данные имеют структурированный вид и поэтому сохраняются в отдельных JSON-файлах. Пользовательские файлы, наоборот, могут иметь любой формат и размер, поэтому они не записываются внутрь JSON-структур, а сохраняются как отдельные объекты в каталогах сервера.

Для хранения служебной информации в проекте предусмотрен каталог *data*. В нём размещаются следующие JSON-файлы: *users.json*, *files.json* и *logs.json*. Файл *users.json* используется для хранения данных о зарегистрированных пользователях. В нём сохраняются идентификатор пользователя, имя, адрес электронной почты, хеш пароля, роль, дата регистрации и признак блокировки. Файл *files.json* содержит метаданные загруженных файлов: идентификатор файла, идентификатор владельца, исходное имя, системное имя, размер, расширение, путь размещения, дату загрузки и дополнительные признаки состояния файла. Файл *logs.json* предназначен для хранения журнала действий пользователей и администратора.

Физическое хранение файлов организовано в отдельном каталоге *storage*. Для каждого пользователя создаётся собственная директория вида *storage/users/user_<id>/*. Такое решение позволяет разделить файлы разных пользователей на уровне файловой системы и упростить контроль доступа. Даже если два пользователя загрузят файлы с одинаковыми названиями, они будут размещены в разных пользовательских каталогах и не будут конфликтовать между собой.

При загрузке файла сервер получает объект из HTML-формы, проверяет его размер и сохраняет в каталог текущего пользователя. Для предотвращения конфликтов имён каждому файлу может присваиваться уникальное системное имя. При этом исходное имя файла сохраняется отдельно в метаданных, чтобы пользователь видел привычное название в интерфейсе. Такой подход позволяет одновременно обеспечить удобство работы для пользователя и безопасность хранения на стороне сервера.

Структура хранения данных проектируется таким образом, чтобы пользователь не работал напрямую с физическими путями файлов. В интерфейсе отображаются только логические сведения: имя файла, размер, тип, дата загрузки, папка, статус избранного или нахождение в корзине. При выполнении операции скачивания, удаления, переименования или перемещения система сначала находит запись о файле в `files.json`, проверяет владельца файла и только после этого выполняет действие с соответствующим объектом файловой системы.

Для реализации работы с JSON-файлами в проекте предусматривается слой репозитория. Универсальный репозиторий выполняет чтение, запись, добавление, обновление и удаление записей. При изменении данных файл перезаписывается в структурированном виде. Такой подход позволяет отделить бизнес-логику приложения от деталей хранения данных. Например, сервис загрузки файла не должен самостоятельно знать, как именно записать данные в JSON-файл: он формирует объект файла и передаёт его в соответствующий репозиторий.

Важной частью проектирования является обеспечение целостности данных. При добавлении нового файла в систему должны быть выполнены две связанные операции: сохранение физического файла в каталог пользователя и добавление записи о нём в `files.json`. Если запись о файле отсутствует, система не сможет корректно отобразить его в интерфейсе. Если же метаданные есть, но физический файл был удалён вручную, скачивание станет невозможным. Поэтому все операции с файлами должны выполняться только через серверную логику приложения.

В системе также предусматривается механизм корзины. При обычном удалении файл не удаляется сразу из файловой системы, а получает специальный признак нахождения в корзине. Благодаря этому пользователь может восстановить файл, если удалил его случайно. Окончательное удаление выполняется отдельно: в этом случае запись удаляется из метаданных, а сам файл удаляется из серверного каталога. Такая логика делает работу с файлами более безопасной и приближает приложение к реальным облачным хранилищам.

Для удобства пользователя в структуре метаданных предусмотрены дополнительные признаки. Например, файл может быть отмечен как избранный, что позволяет быстро найти важные документы. Также для каждого файла сохраняется расширение, на основании которого система может определить тип объекта: документ, изображение, архив, видео, аудио или другой файл. Эти данные используются при отображении списка файлов, поиске и сортировке.

Проектируемая структура хранения поддерживает ограничение пользовательской квоты. Для этого система может подсчитывать общий размер всех активных файлов пользователя и сравнивать его с максимально допустимым объёмом. Если лимит превышен, загрузка нового файла запрещается, а пользователю выводится соответствующее сообщение. Такая функция позволяет контролировать использование серверного пространства и предотвращать бесконтрольное заполнение хранилища.

Таким образом, выбранная схема хранения данных обеспечивает простую и понятную организацию файлового хранилища. JSON-файлы используются для сохранения структурированных метаданных, а файловая система сервера – для размещения реальных пользовательских файлов. Разделение данных по пользователям, использование уникальных имён файлов, проверка владельца и наличие корзины позволяют реализовать базовые механизмы безопасного облачного хранилища в рамках учебного веб-приложения CloudBox.

2.3 Проектирование пользовательского интерфейса

При проектировании пользовательского интерфейса веб-приложения CloudBox основной целью являлось создание понятной и удобной среды для работы с файлами. Так как приложение предназначено для использования через браузер, интерфейс должен обеспечивать быстрый доступ к основным функциям файлового хранилища без необходимости обращения пользователя к внутренней структуре серверных каталогов. Пользователь взаимодействует только с визуальными элементами: формами, кнопками, списками файлов, папками и панелью действий.

Интерфейс приложения проектировался с учётом разделения пользователей по ролям. Для неавторизованного пользователя доступны только страницы регистрации и входа в систему. После успешной авторизации обычный пользователь получает доступ к личному кабинету, где отображается содержимое его файлового пространства. Администратор после входа

получает доступ к административной панели, предназначенной для контроля пользователей, просмотра статистики и анализа журнала действий.

Страница регистрации предназначена для создания новой учётной записи. На ней пользователь вводит имя, адрес электронной почты и пароль. При проектировании данной страницы важно было предусмотреть простоту заполнения формы и понятную обработку ошибок. Если пользователь оставляет обязательные поля пустыми, вводит некорректный email или пытается зарегистрироваться с уже существующим адресом, система должна вывести соответствующее сообщение и не создавать повторную запись.

Страница авторизации используется для входа зарегистрированного пользователя в систему. Она содержит поля для ввода email и пароля. После отправки формы сервер проверяет введённые данные, и при успешной проверке пользователь перенаправляется в личный кабинет. Если данные введены неверно или пользователь заблокирован, на странице отображается сообщение об ошибке. Такой подход позволяет сделать процесс входа понятным и контролируемым.

Главным элементом пользовательской части является личный кабинет. Он представляет собой страницу, на которой пользователь видит свои файлы и папки. В верхней части интерфейса размещается панель управления, содержащая кнопку загрузки файла, кнопку создания папки, строку поиска и элементы сортировки. Основная область страницы предназначена для отображения файлового списка. Для каждого файла выводятся название, тип, размер, дата загрузки и доступные действия.

При проектировании списка файлов было важно сделать управление похожим на привычный файловый менеджер. Пользователь должен быстро понимать, какие объекты являются файлами, а какие – папками. Для этого в интерфейсе могут использоваться иконки, разные подписи и дополнительные признаки. При переходе в папку содержимое страницы обновляется, а пользователь видит текущий путь.

Для каждого файла предусматривается набор действий. Пользователь может скачать файл, переименовать его, переместить в другую папку, отметить как избранный или удалить. Удаление реализуется через корзину, поэтому при первом удалении файл не исчезает окончательно, а переносится в специальный раздел. Это снижает риск случайной потери данных и делает приложение более удобным для пользователя.

Отдельная страница или режим просмотра предназначены для корзины. В корзине отображаются файлы, которые были удалены пользователем, но ещё не удалены окончательно. Для таких файлов доступны действия восстановления и окончательного удаления. При восстановлении файл возвращается в основное хранилище пользователя. При окончательном

удалении запись о файле удаляется из метаданных, а физический файл удаляется из файловой системы сервера.

В приложении также предусмотрен раздел избранных файлов. Он предназначен для быстрого доступа к наиболее важным объектам. Пользователь может отметить файл как избранный, после чего он будет отображаться в соответствующем разделе. Такая функция особенно полезна при большом количестве файлов, когда требуется быстро найти часто используемые документы.

Для повышения удобства работы в интерфейсе проектируется поиск по названию файла. Пользователь вводит часть имени в поисковую строку, после чего система отображает подходящие результаты. Дополнительно предусматривается сортировка файлов по имени, дате загрузки, размеру и типу.

Административный интерфейс отличается от пользовательского. Он предназначен не для работы с личными файлами, а для контроля состояния системы. В административной панели должны отображаться список пользователей, количество зарегистрированных аккаунтов, количество файлов, общий объём занятого пространства и журнал действий. Для каждого пользователя администратор может выполнить блокировку, разблокировку или удаление. Это позволяет управлять доступом к системе и контролировать её использование.

При проектировании интерфейса учитывалась необходимость вывода информационных сообщений. После выполнения действий система должна сообщать пользователю о результате операции. Например, после успешной загрузки файла выводится сообщение о завершении загрузки, при превышении квоты – предупреждение, при ошибке входа – уведомление о неверных данных.

Визуальная часть приложения должна быть выполнена в едином стиле. Все страницы должны иметь общую структуру: заголовок, основную область, панель действий и блок сообщений. Для этого в проекте используется базовый HTML-шаблон, от которого наследуются остальные страницы. Такой подход уменьшает дублирование кода и обеспечивает единообразное оформление интерфейса.

Пользовательский интерфейс CloudBox проектируется как простой веб-файловый менеджер, доступный через браузер. Он должен обеспечивать регистрацию, вход в систему, загрузку и управление файлами, работу с папками, корзиной, избранным, поиском и сортировкой. Административная часть дополняет систему средствами контроля пользователей и просмотра статистики. Такое построение интерфейса делает приложение удобным для работы и соответствует назначению облачного файлового хранилища.

3 РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

3.1 Реализация серверной части веб-приложения

Серверная часть веб-приложения CloudBox реализована на языке программирования Python с использованием микрофреймворка Flask. Данный фреймворк был выбран, так как он позволяет создавать веб-приложения с понятной структурой маршрутов, поддерживает работу с HTML-шаблонами, пользовательскими сессиями, обработкой форм и загрузкой файлов. Flask также хорошо подходит для учебных проектов, поскольку не требует сложной начальной конфигурации и позволяет явно показать логику обработки HTTP-запросов.

Основным файлом запуска приложения является `app.py`. В нём создаётся объект Flask-приложения, загружается конфигурация, инициализируются репозитории и сервисы, а также регистрируются основные группы маршрутов. Для разделения логики используются `blueprints`: отдельный модуль отвечает за регистрацию и авторизацию пользователей, отдельный – за работу с файлами, отдельный – за административную панель. Такое разделение делает структуру проекта более понятной и упрощает дальнейшее сопровождение программы.

Конфигурационные параметры приложения вынесены в файл `config.py`. В нём задаются пути к каталогам хранения данных, имена JSON-файлов, ограничения на размер загружаемых файлов и пользовательская квота. При запуске приложения выполняется проверка наличия необходимых каталогов и файлов. Если каталог `data`, каталог `storage` или JSON-файлы отсутствуют, они создаются автоматически. Это позволяет запускать приложение без ручной подготовки структуры хранения.

Серверная часть приложения построена по модульному принципу. В проекте выделяются маршруты, сервисы, репозитории и модели. Маршруты принимают HTTP-запросы от клиента и передают данные во внутреннюю логику приложения. Сервисы выполняют основные действия: регистрацию, авторизацию, загрузку файлов, работу с папками, поиск, сортировку, удаление, восстановление и подсчёт статистики. Репозитории отвечают за чтение и запись данных в JSON-файлы. Модели описывают основные сущности системы: пользователя, файл и запись журнала.

Для обработки пользовательских запросов применяются HTTP-методы GET и POST. GET-запросы используются для открытия страниц, просмотра личного кабинета, корзины, избранного и административной панели. POST-запросы применяются для действий, изменяющих состояние системы: регистрации, входа, загрузки файла, создания папки, удаления,

восстановления, переименования и перемещения файлов. Такое разделение соответствует общему принципу построения веб-приложений, при котором получение данных и изменение данных выполняются разными типами запросов.

В серверной части реализована система регистрации и авторизации пользователей. При регистрации пользователь вводит данные учётной записи, после чего сервер выполняет проверку корректности введённой информации. Проверяется заполненность обязательных полей, формат адреса электронной почты и отсутствие пользователя с таким же email. Пароль не сохраняется в открытом виде: перед записью в JSON-файл он преобразуется в хеш с использованием средств библиотеки Werkzeug. Это повышает безопасность приложения и исключает хранение паролей в исходном виде.

После успешной авторизации данные пользователя сохраняются в сессии. Благодаря этому сервер может определять текущего пользователя при переходе между страницами и выполнении действий с файлами. Для защиты закрытых маршрутов используются специальные декораторы. Декоратор *login_required* запрещает доступ к личному кабинету и файловым операциям неавторизованным пользователям. Декоратор *admin_required* ограничивает доступ к административной панели и разрешает его только пользователям с ролью администратора.

Работа с файлами реализована в сервисе файлового хранилища. При загрузке файла сервер получает объект из HTML-формы, проверяет его размер, определяет расширение и категорию, формирует уникальное системное имя и сохраняет файл в каталог текущего пользователя. Одновременно с этим создаётся запись с метаданными, которая сохраняется в *files.json*. В метаданные входят исходное имя файла, системное имя, размер, расширение, путь размещения, дата загрузки, идентификатор владельца и дополнительные признаки состояния файла.

Для каждого пользователя создаётся отдельная директория в файловой системе сервера. Это позволяет логически разделить данные разных пользователей и исключить смешивание файлов. При выполнении операций скачивания, удаления, переименования или перемещения сервер сначала проверяет, принадлежит ли выбранный файл текущему пользователю. Если файл принадлежит другому пользователю или не найден, операция не выполняется. Такая проверка предотвращает несанкционированный доступ к чужим данным.

В приложении реализована работа с папками. Пользователь может создавать папки, переходить между ними и размещать в них файлы. Путь к папке сохраняется в метаданных, а фактическая структура создаётся в файловой системе сервера. Это позволяет приблизить интерфейс приложения

к обычному файловому менеджеру и сделать работу с хранилищем более привычной для пользователя.

Удаление файлов реализовано через механизм корзины. При обычном удалении файл не удаляется окончательно, а получает специальный признак нахождения в корзине. Благодаря этому пользователь может восстановить случайно удалённый файл. Окончательное удаление выполняется отдельным действием: в этом случае запись удаляется из JSON-файла, а физический файл удаляется из серверного каталога. Такой подход повышает надёжность работы приложения и снижает риск потери данных.

Дополнительно в серверной части реализованы поиск и сортировка файлов. Поиск выполняется по имени файла и позволяет быстро находить нужные объекты в личном хранилище. Сортировка может выполняться по названию, дате загрузки, размеру или типу файла. Эти функции особенно полезны при большом количестве загруженных объектов, так как позволяют пользователю быстрее ориентироваться в содержимом хранилища.

Для контроля действий пользователей используется журналирование. При выполнении значимых операций, таких как регистрация, вход, загрузка, удаление, восстановление или блокировка пользователя, в журнал добавляется новая запись. Журнал хранится в файле `logs.json` и может просматриваться администратором. Это позволяет отслеживать активность пользователей и анализировать работу системы.

В административной части серверной логики реализованы функции просмотра списка пользователей, блокировки, разблокировки и удаления учётных записей. Также администратору доступна статистика системы: количество пользователей, количество файлов и общий объём занятого пространства. Эти сведения формируются на основе данных из JSON-файлов и файловой системы сервера.

Для обработки ошибок в приложении предусмотрены специальные обработчики. Если пользователь пытается открыть несуществующую страницу, сервер возвращает страницу ошибки 404. Если размер загружаемого файла превышает установленное ограничение, обрабатывается ошибка 413. Вместо аварийного завершения работы приложение выводит пользователю понятное сообщение. Это делает систему более устойчивой и удобной для использования.

Таким образом, серверная часть CloudBox обеспечивает обработку HTTP-запросов, авторизацию пользователей, разграничение доступа, управление файлами, работу с JSON-хранилищем, ведение журнала действий и выполнение административных функций. Реализованная модульная структура позволяет отделить маршруты от бизнес-логики и хранения данных,

что делает приложение более понятным, расширяемым и удобным для дальнейшей доработки.

3.2 Реализация механизмов получения и обработки данных

Основной функциональной частью веб-приложения CloudBox является механизм хранения и обработки пользовательских файлов. Данный модуль обеспечивает загрузку файлов на сервер, сохранение сведений о них, отображение файлов в личном кабинете пользователя, скачивание, переименование, перемещение, удаление, восстановление из корзины, поиск и сортировку. Реализация этих возможностей позволяет рассматривать приложение как полноценное учебное облачное файловое хранилище.

Хранение данных в приложении разделено на две части. Сами файлы сохраняются в файловой системе сервера, а сведения о них записываются в JSON-файл. Такой подход позволяет не смешивать содержимое пользовательских файлов со служебной информацией. Физические файлы могут иметь любой формат и размер в пределах установленного ограничения, а JSON-файл содержит только структурированные метаданные, необходимые для отображения и управления файлами через веб-интерфейс.

При загрузке файла пользователь выбирает объект на своём устройстве и отправляет его через HTML-форму. Браузер формирует HTTP-запрос к серверу, после чего Flask-приложение принимает файл и передаёт его в сервисный слой. На этом этапе выполняется проверка корректности операции: определяется наличие файла в запросе, проверяется его имя, размер и доступное место в пользовательском хранилище. Если файл превышает допустимый объём или пользовательская квота уже заполнена, загрузка не выполняется, а пользователю выводится сообщение об ошибке.

Для каждого пользователя в файловой системе сервера создаётся отдельный каталог. Это позволяет разделить данные разных пользователей и исключить ситуацию, при которой файлы одного пользователя будут отображаться или изменяться другим пользователем. При сохранении файла сервер использует не только исходное имя, но и внутреннее системное имя. Такой подход необходим для предотвращения конфликтов, если несколько файлов имеют одинаковое название. При этом в интерфейсе пользователь видит исходное имя файла, что делает работу с хранилищем привычной и понятной.

После физического сохранения файла на сервере формируется запись с его метаданными. В неё входят идентификатор файла, идентификатор владельца, исходное имя, системное имя, расширение, размер, категория, путь размещения, дата загрузки, папка, а также признаки избранного и нахождения

в корзине. Эти сведения сохраняются в файле `files.json`. Благодаря этому приложение может быстро отображать список файлов пользователя без необходимости каждый раз полностью анализировать содержимое серверных каталогов.

Работа с метаданными реализована через репозиторий файлов. Репозиторий отвечает за чтение данных из JSON-файла, добавление новых записей, обновление существующих сведений и удаление записей при окончательном удалении файла. Такое разделение позволяет не размещать операции чтения и записи JSON непосредственно в маршрутах Flask. Маршруты только принимают запрос пользователя, а работа с данными выполняется через отдельные классы репозитория и сервиса.

В приложении реализована поддержка папок. Пользователь может создавать папки и размещать в них файлы. При создании папки в файловой системе формируется соответствующий каталог, а при отображении личного кабинета учитывается текущий путь пользователя. Это позволяет организовать файлы не только в виде общего списка, но и в виде структуры, похожей на обычный файловый менеджер. Пользователь может переходить между папками и работать с файлами в выбранном разделе хранилища.

Для скачивания файла сервер сначала находит запись о нём в метаданных, после чего проверяет владельца файла. Если файл принадлежит текущему пользователю, приложение формирует ответ с передачей файла браузеру. Если файл не найден или не принадлежит пользователю, операция блокируется. Такая проверка необходима для защиты данных, так как пользователь не должен получить доступ к чужому файлу путём ручного изменения адреса страницы или идентификатора объекта.

Переименование файла выполняется через изменение его отображаемого имени в метаданных. Пользователь вводит новое имя, после чего сервер проверяет корректность введённых данных и обновляет соответствующую запись в `files.json`. При этом системное имя файла может оставаться прежним, так как оно используется для безопасного хранения на сервере. Такой подход упрощает переименование и снижает риск ошибок при работе с физическим файлом.

Перемещение файла между папками реализуется путём изменения логического пути и, при необходимости, физического расположения файла в каталоге пользователя. Перед выполнением операции сервер проверяет, существует ли выбранный файл, принадлежит ли он текущему пользователю и доступна ли папка назначения. После успешной проверки данные обновляются, а пользователь видит файл уже в новом расположении. Это позволяет организовать личное хранилище более гибко.

Удаление файлов в приложении реализовано в два этапа. При обычном удалении файл не удаляется сразу из файловой системы, а получает признак нахождения в корзине. Такие файлы перестают отображаться в основном списке, но доступны в отдельном разделе корзины. Пользователь может восстановить файл, если удаление было выполнено случайно. При восстановлении признак корзины снимается, и файл снова становится доступен в личном хранилище.

Окончательное удаление выполняется отдельно. В этом случае запись о файле удаляется из JSON-файла, а сам файл удаляется из файловой системы сервера. Такой механизм делает работу с файлами более безопасной, так как пользователь получает возможность исправить ошибочное удаление до окончательной очистки. Подобная логика используется во многих файловых менеджерах и облачных хранилищах.

Дополнительно в CloudBox реализована возможность добавления файлов в избранное. Для этого в метаданных файла хранится специальный логический признак. Если пользователь отмечает файл как избранный, он может быстро найти его в отдельном разделе. Эта функция удобна при большом количестве файлов, так как позволяет выделить наиболее важные документы, изображения или другие материалы.

Для удобной работы с хранилищем реализован поиск по названию файла. Пользователь вводит часть имени, после чего приложение отбирает только те записи, которые соответствуют поисковому запросу. Поиск выполняется по метаданным, что позволяет не обращаться напрямую к содержимому файлов. Такой способ является быстрым и достаточным для учебного файлового хранилища.

Также предусмотрена сортировка файлов. Пользователь может упорядочить список по имени, размеру, дате загрузки или типу файла. Для этого сервер получает параметр сортировки, выбирает соответствующее поле метаданных и формирует список в нужном порядке. Сортировка повышает удобство работы с большим количеством файлов и делает интерфейс более похожим на полноценный файловый менеджер.

Важной частью обработки файлов является определение их категории. Категория может определяться по расширению файла. Например, изображения, документы, архивы, аудио- и видеофайлы могут отображаться в интерфейсе по-разному. Это упрощает восприятие списка файлов и помогает пользователю быстрее ориентироваться в содержимом хранилища.

В системе также реализован контроль пользовательской квоты. Перед загрузкой нового файла сервер подсчитывает общий объем файлов пользователя и сравнивает его с максимально допустимым значением. Если добавление нового файла приводит к превышению квоты, операция не

выполняется. Это позволяет ограничить использование дискового пространства сервера и предотвращает неконтролируемое заполнение хранилища.

Таким образом, механизм хранения и обработки файлов в CloudBox обеспечивает полный набор базовых операций, необходимых для работы облачного файлового хранилища. Пользователь может загружать файлы, просматривать их, скачивать, переименовывать, перемещать, удалять, восстанавливать, искать, сортировать и добавлять в избранное. Разделение физических файлов и JSON-метаданных делает систему простой, понятной и удобной для сопровождения, а проверка владельца файла и использование пользовательских каталогов обеспечивают базовую защиту данных.

3.3 Реализация системы авторизации и персонализации пользователя

В веб-приложении CloudBox реализована система авторизации и разграничения прав доступа, предназначенная для защиты пользовательских данных и ограничения доступа к функциям приложения в зависимости от роли пользователя. Наличие такого механизма является обязательным для файлового хранилища, так как каждый пользователь должен иметь доступ только к собственным файлам, папкам и настройкам.

Работа с пользователями начинается с процедуры регистрации. Новый пользователь заполняет регистрационную форму, указывая необходимые данные для создания учётной записи. После отправки формы серверная часть приложения проверяет корректность введённой информации. В частности, выполняется проверка заполненности обязательных полей, корректности адреса электронной почты и отсутствия уже зарегистрированного пользователя с таким же email. Если данные не проходят проверку, пользователь получает сообщение об ошибке, а учётная запись не создаётся.

При успешной регистрации создаётся новая запись пользователя, которая сохраняется в файле `users.json`. Для каждого пользователя формируется уникальный идентификатор, сохраняется email, имя пользователя, роль, дата регистрации и служебные признаки, необходимые для работы системы. Такой подход позволяет хранить данные пользователей в структурированном виде без использования внешней базы данных.

Особое внимание уделено хранению паролей. В приложении пароль пользователя не записывается в JSON-файл в открытом виде. Перед сохранением он преобразуется в хеш с использованием средств библиотеки Werkzeug. При последующем входе в систему введённый пользователем пароль также проверяется через механизм сравнения с хешем. Это позволяет повысить безопасность приложения, так как даже при просмотре файла с пользователями невозможно получить исходные пароли.

Процедура авторизации выполняется через отдельную форму входа. Пользователь вводит email и пароль, после чего сервер обращается к сервису авторизации. Сервис выполняет поиск пользователя по email и проверяет соответствие введённого пароля сохранённому хешу. Если данные указаны верно, пользователь считается авторизованным, а информация о нём сохраняется в пользовательской сессии. Если email или пароль введены неправильно, сервер возвращает пользователя на страницу входа и выводит сообщение об ошибке.

Для хранения состояния входа используется механизм сессий Flask. После успешной авторизации в сессию записывается идентификатор пользователя и его роль. Благодаря этому при переходе между страницами приложение может определить, какой пользователь выполняет запрос. Это необходимо для отображения личного кабинета, проверки доступа к файлам и выполнения действий только от имени текущего пользователя.

В системе предусмотрены две основные роли: обычный пользователь и администратор. Обычный пользователь получает доступ к личному файловому хранилищу. Он может загружать файлы, создавать папки, скачивать, переименовывать, перемещать, удалять и восстанавливать только собственные файлы. Пользователь не имеет доступа к данным других учётных записей и не может открывать административные страницы.

Администратор имеет расширенные права. После входа в систему он получает доступ к административной панели, где может просматривать список пользователей, статистику файлового хранилища и журнал действий. Также администратор может блокировать, разблокировать и удалять пользователей. Такое разделение прав позволяет контролировать работу системы и управлять доступом к приложению.

Для защиты маршрутов используются специальные декораторы. Декоратор `login_required` применяется к страницам, которые должны быть доступны только авторизованным пользователям. Если неавторизованный пользователь попытается открыть личный кабинет или выполнить действие с файлом, система перенаправит его на страницу входа. Декоратор `admin_required` применяется к административным маршрутам и дополнительно проверяет роль пользователя. Если обычный пользователь попытается перейти в административную панель, доступ будет запрещён.

Кроме проверки авторизации, в приложении реализована проверка принадлежности файлов. При скачивании, переименовании, перемещении, удалении или восстановлении файла сервер сначала получает сведения о файле из `files.json`, а затем сравнивает идентификатор владельца файла с идентификатором текущего пользователя. Если файл не принадлежит пользователю, операция не выполняется. Такой механизм защищает данные от несанкционированного доступа даже в том случае, если пользователь вручную изменит адрес страницы или идентификатор файла в запросе.

Также в системе предусмотрена возможность блокировки пользователя. Если администратор блокирует учётную запись, пользователь теряет возможность входа в систему. При попытке авторизации приложение проверяет признак блокировки и не допускает пользователя к личному кабинету. Такая функция может использоваться для ограничения доступа при нарушении правил работы с системой или при необходимости временно отключить учётную запись.

После выхода из системы данные авторизации удаляются из сессии. Пользователь больше не может выполнять защищённые действия, пока снова не войдёт в приложение. Это особенно важно при работе за общим компьютером, так как завершение сеанса предотвращает доступ посторонних лиц к личному файловому хранилищу.

Важной частью системы безопасности является обработка некорректных запросов. Если пользователь пытается открыть страницу, к которой у него нет доступа, выполнить действие с чужим файлом или обратиться к несуществующему объекту, приложение не должно завершаться с ошибкой. Вместо этого пользователь получает понятное сообщение или перенаправляется на доступную страницу. Такой подход делает работу приложения более устойчивой и безопасной.

Таким образом, реализованная система авторизации и разграничения прав доступа обеспечивает защиту пользовательских данных в CloudBox. Использование хеширования паролей, сессий Flask, ролей пользователей, декораторов доступа и проверки владельца файла позволяет создать базовый уровень безопасности, необходимый для веб-приложения облачного файлового хранилища.

3.4 Реализация клиентской части веб-приложения

Клиентская часть веб-приложения CloudBox реализована в виде HTML-страниц, которые отображаются в браузере пользователя. Она предназначена для удобного взаимодействия с файловым хранилищем и предоставляет доступ к основным функциям системы: регистрации, авторизации, загрузке файлов, просмотру содержимого хранилища, поиску, сортировке и управлению файлами.

Для формирования страниц используется шаблонизатор Jinja2, который применяется во Flask для динамической генерации HTML-кода. С его помощью сервер передаёт в шаблоны данные о текущем пользователе, списке файлов, папках, результатах поиска, занятом объёме хранилища и системных сообщениях. Благодаря этому пользователь видит актуальное состояние своего файлового пространства после каждого выполненного действия.

В проекте используется общий базовый шаблон, на основе которого строятся остальные страницы приложения. В нём размещаются основные элементы интерфейса: заголовок, навигационная панель, область вывода уведомлений и основная рабочая область. Такой подход позволяет сохранить единый стиль оформления на всех страницах и избежать дублирования одинаковых фрагментов HTML-кода.

Страница регистрации предназначена для создания новой учётной записи. Пользователь вводит необходимые данные, после чего форма отправляется на сервер для проверки. Если данные заполнены корректно и пользователь с таким email ещё не существует, создаётся новая учётная запись. При наличии ошибок система выводит соответствующее сообщение, позволяя пользователю исправить введённые данные.

Страница авторизации обеспечивает вход пользователя в систему. Она содержит поля для ввода email и пароля. После отправки формы сервер проверяет учётные данные и при успешной авторизации перенаправляет пользователя в личный кабинет. Если данные введены неверно или учётная запись заблокирована, пользователю отображается сообщение об ошибке.

Основной страницей пользовательской части является личный кабинет. В нём отображается содержимое файлового хранилища текущего пользователя. Пользователь видит список файлов и папок, их названия, размеры, типы и даты загрузки. Для каждого файла предусмотрены действия, позволяющие скачать, переименовать, переместить, удалить или добавить объект в избранное.

В верхней части личного кабинета размещается панель управления. Она содержит форму загрузки файла, кнопку создания папки, строку поиска и элементы сортировки. Через форму загрузки пользователь выбирает файл на своём устройстве и отправляет его на сервер. Кнопка создания папки используется для добавления нового каталога в текущем разделе хранилища. Поисковая строка позволяет быстро найти нужный файл по части его имени.

Для удобства работы с каталогами в интерфейсе отображается текущая папка. Пользователь может переходить между папками и возвращаться на предыдущий уровень. Это делает взаимодействие с приложением похожим на работу с обычным файловым менеджером. При переходе в другую папку сервер формирует новый список объектов и передаёт его в шаблон для отображения.

Отдельно реализован интерфейс корзины. В этом разделе отображаются файлы, которые были удалены пользователем, но ещё не удалены окончательно. Для таких объектов доступны функции восстановления и полного удаления. При восстановлении файл возвращается в основное хранилище, а при окончательном удалении полностью удаляется из системы.

Также в клиентской части предусмотрен раздел избранных файлов. В него попадают объекты, которые пользователь отметил как важные. Такая возможность позволяет быстрее находить часто используемые документы, изображения или другие файлы. В общем списке избранные объекты могут выделяться специальной отметкой, что упрощает их визуальное распознавание.

Поиск файлов реализован через отдельную строку ввода. Пользователь вводит часть названия файла, после чего система отображает подходящие результаты. Это особенно удобно при большом количестве загруженных объектов. Кроме поиска, реализована сортировка файлов по имени, размеру, дате загрузки и типу. Благодаря этому пользователь может быстро упорядочить содержимое хранилища нужным способом.

Для информирования пользователя о результатах операций используются системные сообщения. После успешной загрузки файла, создания папки, удаления, восстановления или переименования система выводит уведомление. Если операция не может быть выполнена, например из-за превышения квоты, отсутствия файла или некорректного ввода, пользователь также получает понятное сообщение об ошибке.

Административная часть интерфейса предназначена для пользователя с ролью администратора. В административной панели отображаются список зарегистрированных пользователей, общая статистика файлового хранилища и журнал действий. Администратор может просматривать информацию о пользователях, блокировать и разблокировать учётные записи, а также удалять пользователей из системы.

Визуальное оформление приложения выполнено средствами HTML и CSS. Элементы интерфейса расположены таким образом, чтобы основные действия были доступны пользователю без лишней навигации. Страницы имеют единый стиль, понятные подписи, логичное расположение кнопок и информационные сообщения, что делает работу с приложением более простой и удобной.

Таким образом, клиентская часть CloudBox обеспечивает полноценное взаимодействие пользователя с файловым хранилищем через браузер. С помощью HTML-шаблонов, форм, таблиц и управляющих элементов реализованы регистрация, авторизация, просмотр файлов, загрузка, скачивание, удаление, восстановление, поиск, сортировка, работа с папками, избранным и административной панелью. Реализованный интерфейс делает приложение понятным и удобным для использования.

4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

Тестирование веб-приложения CloudBox выполнялось с целью проверки корректности работы основных функций системы, механизмов авторизации, обработки HTTP-запросов и операций с пользовательскими файлами. В ходе тестирования проверялось, что приложение корректно принимает запросы от браузера, обрабатывает действия пользователя на стороне Flask-сервера, сохраняет файлы в файловой системе и обновляет метаданные в JSON-файлах.

Для анализа сетевого взаимодействия использовалась программа Wireshark. Приложение было запущено на локальном сервере, доступ к которому выполнялся по IP-адресу и порту сервера. В Wireshark применялся фильтр по порту приложения, что позволило отобразить только те пакеты, которые относятся к работе разрабатываемого веб-приложения. Такой способ проверки позволяет наглядно подтвердить, что взаимодействие между браузером и сервером выполняется по протоколу HTTP.

Первым этапом было проверено открытие страницы авторизации. При переходе пользователя на страницу входа браузер отправляет HTTP-запрос методом GET к маршруту /login. Сервер принимает запрос и возвращает HTML-страницу с формой авторизации. В Wireshark был зафиксирован запрос GET /login HTTP/1.1, а также ответ сервера HTTP/1.1 200 OK, что подтверждает успешную передачу страницы входа клиенту.

106	91.206911	192.168.0.106	192.168.0.104	HTTP	835 GET /login HTTP/1.1
107	91.206960	192.168.0.104	192.168.0.106	TCP	54 5001 → 51025 [ACK] Seq=1 Ack=782 Win=261376 Len=0
108	91.209216	192.168.0.104	192.168.0.106	TCP	333 5001 → 51025 [PSH, ACK] Seq=1 Ack=782 Win=262144 Len=279 [TCP PDU reassembled in 2110]
109	91.209259	192.168.0.104	192.168.0.106	TCP	1514 5001 → 51025 [ACK] Seq=280 Ack=782 Win=262144 Len=1460 [TCP PDU reassembled in 2110]
110	91.209270	192.168.0.104	192.168.0.106	HTTP	761 HTTP/1.1 200 OK (text/html)

Рисунок 4.1 – HTTP-запрос на открытие страницы авторизации.

Следующим этапом проверялась авторизация пользователя. После ввода email и пароля браузер отправляет на сервер данные формы входа. В Wireshark был зафиксирован запрос, содержащий данные в формате *application/x-www-form-urlencoded*. В составе формы передаются поля *email* и *password*. После обработки запроса сервер возвращает ответ со статусом 302 FOUND, что означает успешную проверку данных и перенаправление пользователя в личный кабинет. При вставке данного скриншота в пояснительную записку значение пароля необходимо скрыть.

2348	105.364992	192.168.0.106	192.168.0.104	HTTP	102 POST /login HTTP/1.1 (application/x-www-form-urlencoded)
2349	105.365151	192.168.0.104	192.168.0.106	TCP	54 [TCP Window Update] 5001 → 47162 [ACK] Seq=1 Ack=1 Win=262144 Len=0
2350	105.365231	192.168.0.104	192.168.0.106	TCP	54 5001 → 47754 [ACK] Seq=1 Ack=711 Win=261440 Len=0
2400	106.035608	192.168.0.104	192.168.0.106	TCP	360 5001 → 47754 [PSH, ACK] Seq=1 Ack=711 Win=262144 Len=306 [TCP PDU rea
2401	106.039777	192.168.0.106	192.168.0.104	TCP	54 47754 → 5001 [FIN, ACK] Seq=711 Ack=307 Win=65024 Len=0
2402	106.039843	192.168.0.104	192.168.0.106	HTTP	261 HTTP/1.1 302 FOUND (text/html)
2403	106.039990	192.168.0.104	192.168.0.106	TCP	54 5001 → 47754 [FIN, ACK] Seq=514 Ack=712 Win=262144 Len=0
2404	106.042295	192.168.0.106	192.168.0.104	TCP	54 47754 → 5001 [RST, ACK] Seq=712 Ack=514 Win=0 Len=0
2405	106.042627	192.168.0.106	192.168.0.104	TCP	54 47754 → 5001 [RST] Seq=712 Win=0 Len=0
> Frame 2348: Packet, 102 bytes on wire (816 bits), 102 bytes captured (816 bits) on interface en0, id 0 > Ethernet II, Src: AzureWaveTec_e2:76:cf (14:13:33:e2:76:cf), Dst: ce:64:cb:11:2f:33 (ce:64:cb:11:2f:33) > Internet Protocol Version 4, Src: 192.168.0.106, Dst: 192.168.0.104 > Transmission Control Protocol, Src Port: 47754, Dst Port: 5001, Seq: 663, Ack: 1, Len: 48 > [2 Reassembled TCP Segments (710 bytes): #2347(662), #2348(48)] > Hypertext Transfer Protocol > HTML Form URL Encoded: application/x-www-form-urlencoded > Form item: "email" = "bomba@gmail.com" > Form item: "password" = "fheQUyDde33BeW2"					00b0 6f 6e 2f 78 2d 77 77 00c0 6c 65 6e 63 6f 64 65 00d0 65 2d 49 6e 73 65 63 00e0 73 74 73 3a 20 31 0d 00f0 6e 74 3a 20 4d 6f 7a 0100 28 57 69 6e 64 6f 77 0110 3b 20 57 69 6e 36 34 0120 70 6c 65 57 65 62 4b 0130 20 28 4b 48 54 4d 4c 0140 63 6b 6f 29 20 43 68 0150 30 2e 30 2e 30 20 53 0160 2e 33 36 20 4f 50 52

Рисунок 4.2 – Передача данных авторизации методом POST

После успешного входа была проверена загрузка личного кабинета пользователя. При переходе в личный кабинет браузер отправляет GET-запрос к маршруту /dashboard. Сервер определяет текущего пользователя по данным сессии, получает список его файлов и папок, формирует HTML-страницу и передаёт её браузеру. В Wireshark был зафиксирован запрос GET /dashboard HTTP/1.1 и ответ HTTP/1.1 200 OK, что подтверждает корректное открытие защищённой страницы после авторизации.

2406	-1089.288061	192.168.0.106	192.168.0.104	HTTP	686 GET /dashboard HTTP/1.1
2413	-1089.251733	192.168.0.104	192.168.0.106	HTTP	1004 HTTP/1.1 200 OK (text/html)

Рисунок 4.3 – Ответ сервера при открытии личного кабинета пользователя.

Особое внимание при тестировании уделялось загрузке файлов, так как данная функция является основной для облачного файлового хранилища. При выборе файла и отправке формы браузер формирует HTTP-запрос методом POST к маршруту /upload. В Wireshark был зафиксирован запрос POST /upload HTTP/1.1, в котором передавался файл формата PDF. Это подтверждает, что файл отправляется с клиентской стороны на сервер через HTTP-запрос. После получения файла сервер проверяет его, сохраняет в каталог пользователя и добавляет сведения о нём в JSON-файл с метаданными.

209	78.860853	127.0.0.1	127.0.0.1	HTTP	9389 POST /upload HTTP/1.1 (application/pdf)
-----	-----------	-----------	-----------	------	--

Рисунок 4.4 – HTTP-запрос загрузки файла на сервер.

Также была проверена функция скачивания файла из хранилища. При нажатии кнопки скачивания браузер отправляет GET-запрос к маршруту вида /download/<id>, где <id> является идентификатором файла. В ходе

тестирования был зафиксирован запрос GET /download/13 HTTP/1.1. После получения такого запроса сервер проверяет, принадлежит ли файл текущему пользователю, и передаёт его браузеру для скачивания. Наличие данного запроса в сетевом трафике подтверждает корректную работу механизма получения файла из серверного хранилища.

66162	547.511715	192.168.0.101	192.168.0.104	HTTP	611	GET /download/13	HTTP/1.1
66211	547.535406	192.168.0.104	192.168.0.101	HTTP	739	HTTP/1.1 200 OK	(application/pdf)

Рисунок 4.5 – HTTP-запрос скачивания файла из хранилища.

Для проверки устойчивости приложения был выполнен переход по несуществующему адресу. В этом случае браузер отправляет запрос к маршруту, который не определён в приложении. Сервер Flask корректно обрабатывает такую ситуацию и возвращает ответ HTTP/1.1 404 NOT FOUND. В Wireshark видно, что вместо аварийного завершения работы сервер формирует HTML-страницу ошибки. Это подтверждает корректную обработку некорректных URL-адресов.

1609	507.772219	127.0.0.1	127.0.0.1	HTTP	1298	HTTP/1.1 404 NOT FOUND (text/html)
1610	507.772242	127.0.0.1	127.0.0.1	TCP	56	62013 → 5002 [ACK] Seq=722 Ack=1439 Win=406912 Len=0 TSval=515969832 TSecr=1312634423
1611	507.772355	127.0.0.1	127.0.0.1	TCP	56	62013 → 5002 [FIN, ACK] Seq=722 Ack=1439 Win=406912 Len=0 TSval=515969832 TSecr=1312634423

Рисунок 4.6 – Обработка запроса к несуществующей странице

Дополнительно было проведено функциональное тестирование основных возможностей веб-приложения. Проверялись регистрация пользователя, вход в систему, открытие личного кабинета, загрузка файла, отображение файла в списке, скачивание, переименование, удаление, восстановление из корзины, поиск, сортировка, работа с избранными файлами и административная панель. Результаты функционального тестирования представлены в таблице 4.1.

Таблица 4.1 – Результаты функционального тестирования веб-приложения CloudBox.

Дополнительно было проведено функциональное тестирование пользовательского интерфейса. Результаты проверки основных функций системы сведены в таблицу 4.1.

Таблица 4.1 – Результаты функционального тестирования

Проверяемая функция	Ожидаемый результат	Фактический результат	Статус
Открытие страницы авторизации	Отображается форма входа	Соответствует	Пройден
Авторизация пользователя	Вход в систему и переход в личный кабинет	Соответствует	Пройден
Открытие личного кабинета	Отображается хранилище текущего пользователя	Соответствует	Пройден
Загрузка файла	Файл сохраняется на сервере, метаданные обновляются	Соответствует	Пройден
Скачивание файла	Пользователь получает выбранный файл	Соответствует	Пройден
Удаление файла	Файл переносится в корзину	Соответствует	Пройден
Восстановление файла	Файл возвращается из корзины	Соответствует	Пройден
Поиск файла	Отображаются подходящие файлы	Соответствует	Пройден
Сортировка файлов	Список упорядочивается по выбранному параметру	Соответствует	Пройден
Обработка несуществующей страницы	Сервер возвращает страницу ошибки 404	Соответствует	Пройден
Работа административной панели	Администратор видит пользователей, статистику и журнал действий	Соответствует	Пройден

5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Для работы с веб-приложением CloudBox необходимо запустить серверную часть приложения и открыть его в браузере. Приложение работает как локальный веб-сервис, поэтому после запуска Flask-сервера пользователь переходит по адресу, указанному в консоли. В ходе тестирования приложение запускалось на локальном сервере по адресу вида `http://127.0.0.1:5002`. После перехода по данному адресу пользователю становится доступен веб-интерфейс файлового хранилища.

Перед началом работы пользователь должен пройти регистрацию или выполнить вход в уже существующую учётную запись. Если пользователь ранее не был зарегистрирован, необходимо открыть страницу регистрации и заполнить предложенную форму. В форме указываются данные, необходимые для создания учётной записи. После отправки формы система проверяет введённую информацию. Если данные заполнены корректно и пользователь с таким email ещё отсутствует в системе, создаётся новая учётная запись.

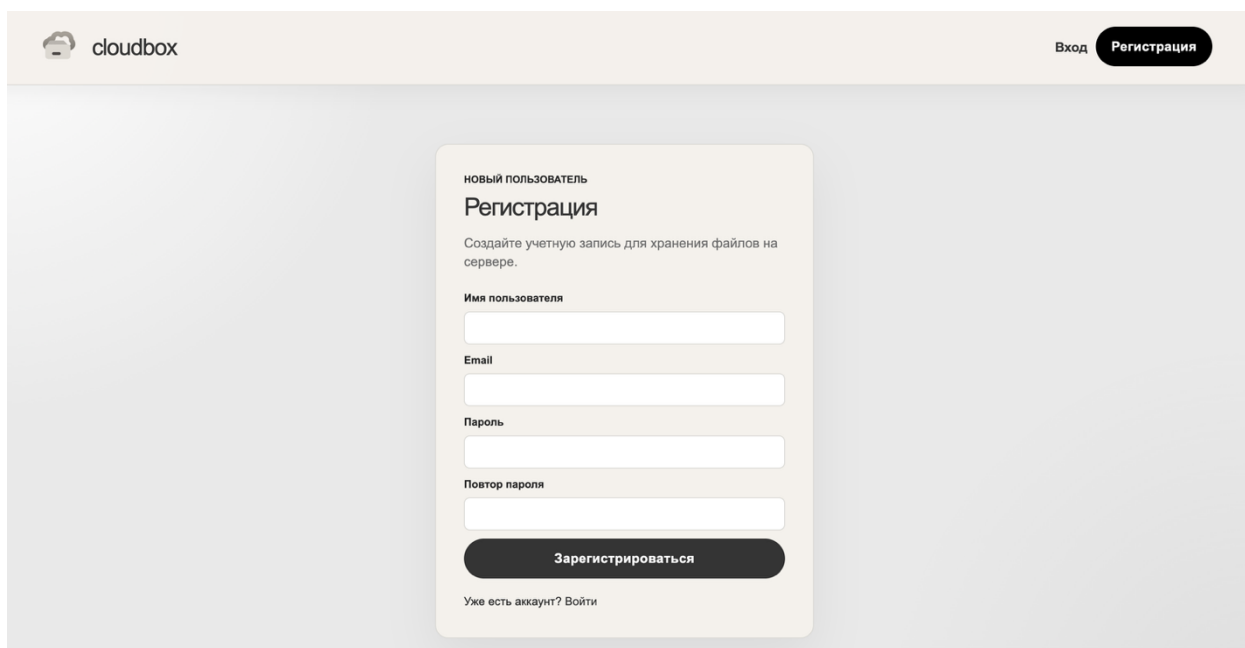


Рисунок 5.1 – Страница регистрации пользователя

После успешной регистрации пользователь может перейти на страницу авторизации. Для входа в систему необходимо указать email и пароль, заданные при регистрации. После нажатия кнопки входа данные отправляются на сервер для проверки. Если email и пароль введены верно, пользователь перенаправляется в личный кабинет. Если данные указаны неправильно или

учётная запись заблокирована администратором, система выводит сообщение об ошибке.

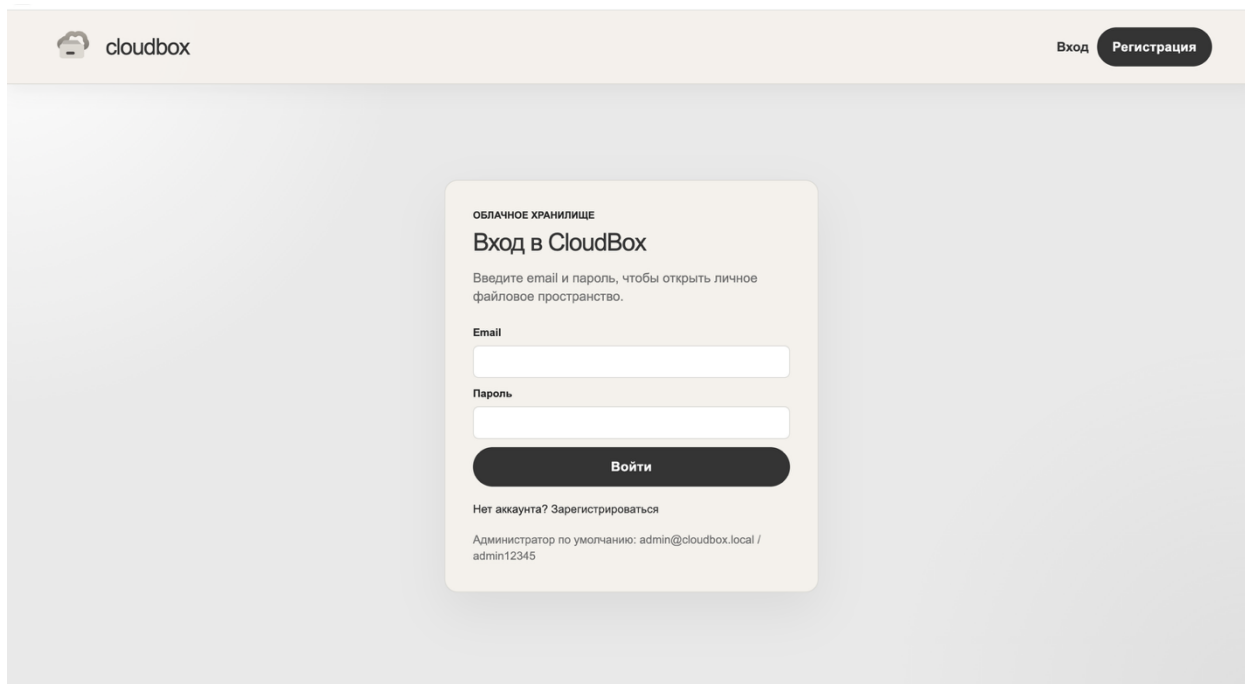


Рисунок 5.2 – Страница авторизации пользователя.

Основной рабочей областью приложения является личный кабинет пользователя. В нём отображается содержимое персонального файлового хранилища. Пользователь видит список своих файлов и папок, а также сведения о каждом объекте: название, тип, размер и дату загрузки. В личном кабинете расположены основные элементы управления: загрузка файла, создание папки, поиск, сортировка и действия над файлами.

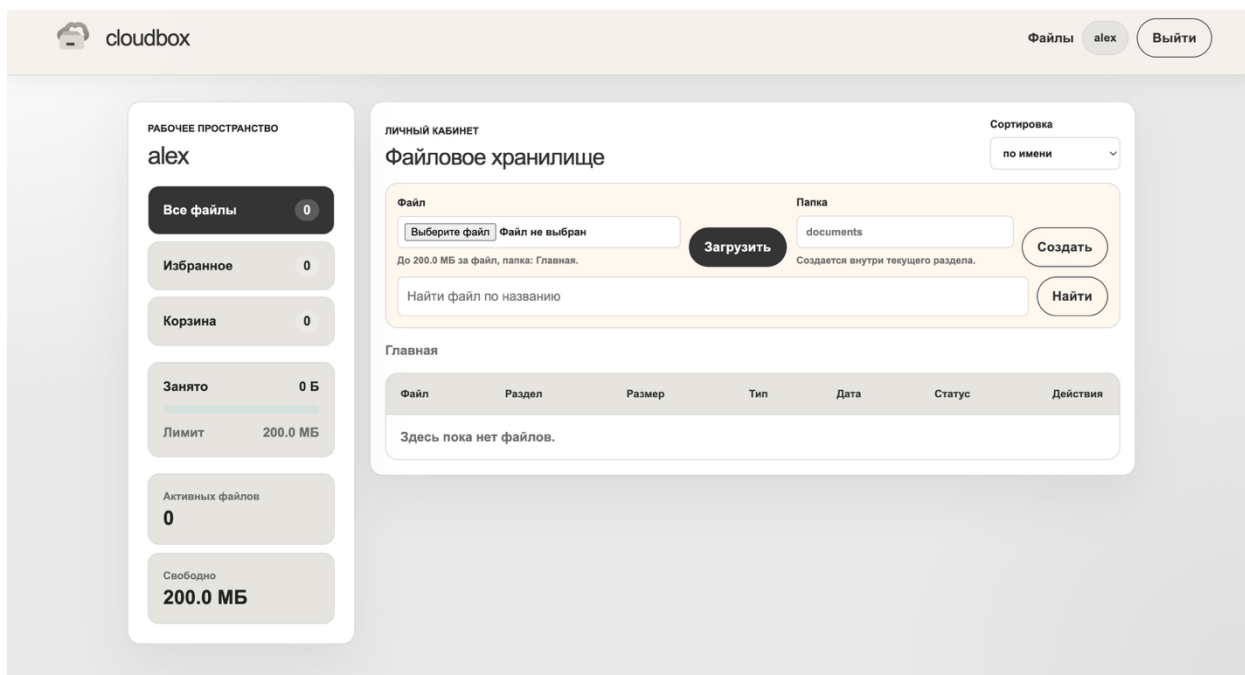


Рисунок 5.3 – Личный кабинет пользователя

Для загрузки файла необходимо нажать кнопку выбора файла, указать нужный объект на компьютере и отправить форму загрузки. После этого файл передаётся на сервер и сохраняется в личном каталоге пользователя. Сведения о загруженном файле записываются в JSON-файл с метаданными. Если файл успешно загружен, он появляется в списке объектов личного кабинета. Если размер файла превышает установленное ограничение или пользовательская квота заполнена, система выводит предупреждающее сообщение.

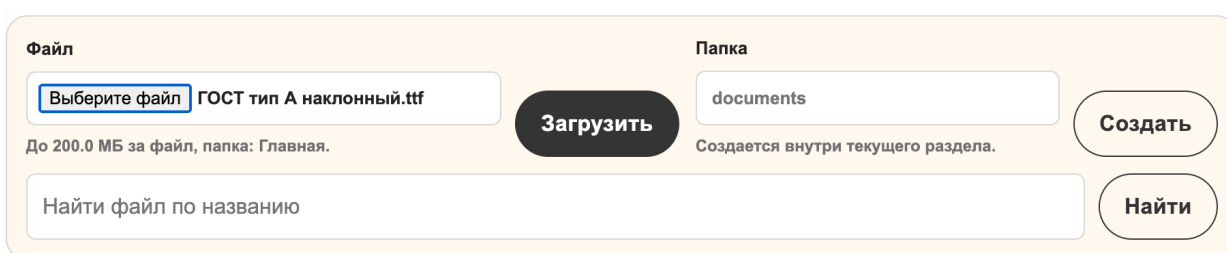


Рисунок 5.4 – Загрузка файла в хранилище.

Для удобной организации данных пользователь может создавать папки. Для этого необходимо нажать кнопку создания папки, ввести её название и подтвердить действие. После создания папка появляется в текущем разделе хранилища. Пользователь может открыть папку и размещать в ней файлы. Навигация между папками выполняется через элементы интерфейса, отображающие текущий путь и доступные каталоги.

Папка

учеба

Создается внутри текущего раздела.

Создать

Рисунок 5.5 – Создание папки в личном хранилище.

Для скачивания файла пользователь нажимает соответствующую кнопку рядом с нужным объектом. После этого браузер отправляет запрос на сервер, а сервер проверяет принадлежность файла текущему пользователю и передаёт файл для скачивания. Если файл найден и доступ разрешён, загрузка начинается автоматически средствами браузера.

Статус	Действия
активен	Скачать Инфо В избранное ГОСТ тип А наклонный. OK Главная папка Переместить В корзину

Рисунок 5.6 – Скачивание файла из хранилища

При необходимости пользователь может переименовать файл. Для этого рядом с выбранным файлом используется действие переименования. Пользователь вводит новое название, после чего система обновляет сведения о файле в метаданных. Переименование меняет отображаемое имя файла в интерфейсе, при этом внутренняя логика хранения остаётся безопасной для серверной файловой системы.

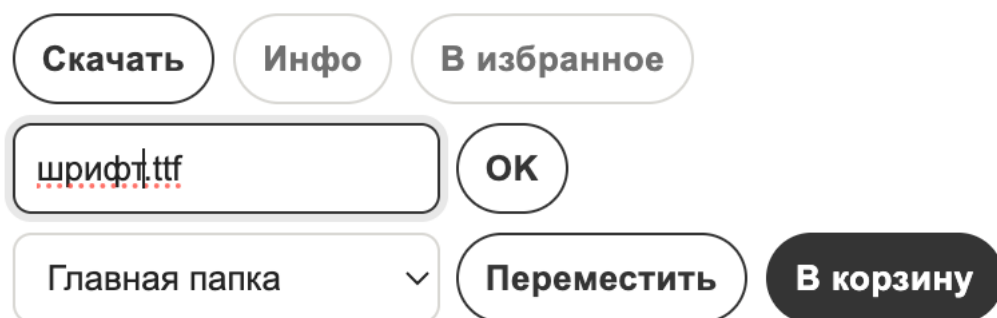


Рисунок 5.7 – Переименование файла

В приложении предусмотрена возможность перемещения файлов между папками. Пользователь выбирает файл и указывает папку назначения. После подтверждения действия файл переносится в выбранный каталог, а сведения о его расположении обновляются в метаданных. Данная функция позволяет поддерживать порядок в личном хранилище и группировать файлы по темам или назначению.

Для быстрого доступа к важным объектам пользователь может отметить файл как избранный. После установки соответствующей отметки файл становится доступен в разделе избранного. Это удобно для документов, изображений или других материалов, к которым пользователь обращается чаще всего.

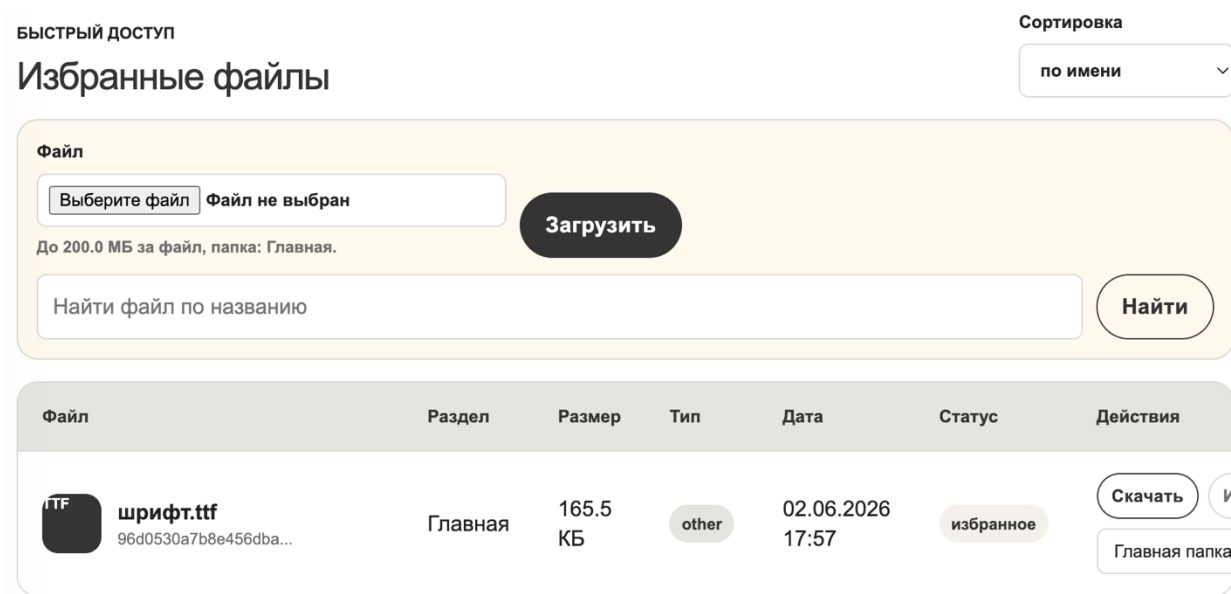


Рисунок 5.8 – Раздел избранных файлов

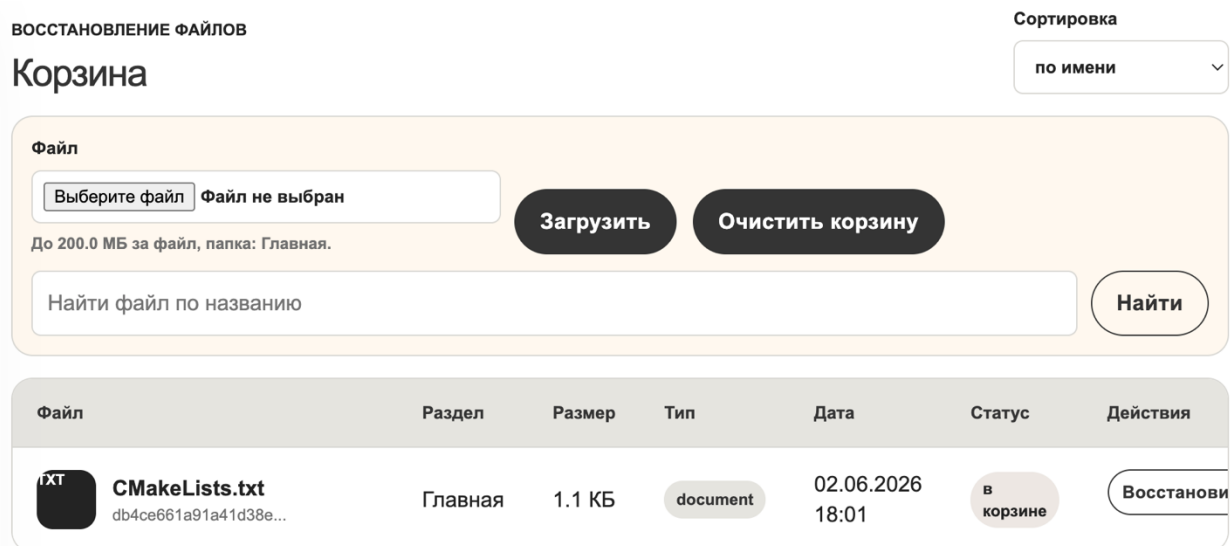


Рисунок 5.9 – Корзина файлового хранилища.

Для поиска файла используется строка поиска. Пользователь вводит часть названия файла, после чего система отображает подходящие результаты. Данная функция полезна при большом количестве загруженных объектов, так как позволяет быстро найти нужный файл без ручного просмотра всего списка.

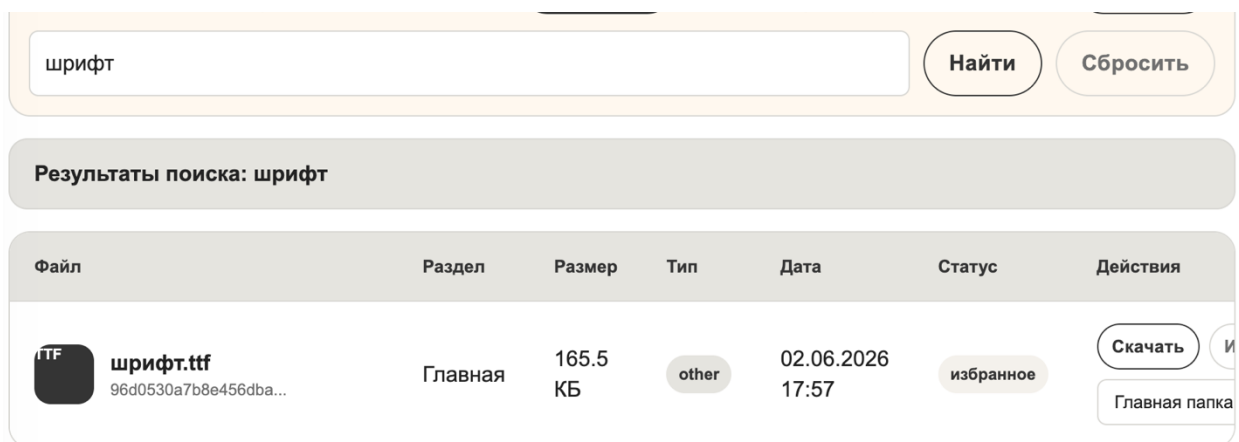


Рисунок 5.10 – Поиск файла по названию.

В личном кабинете также предусмотрена сортировка файлов. Пользователь может упорядочить список по имени, размеру, дате загрузки или типу файла. Сортировка помогает быстрее ориентироваться в содержимом хранилища и удобна при работе с большим количеством документов.

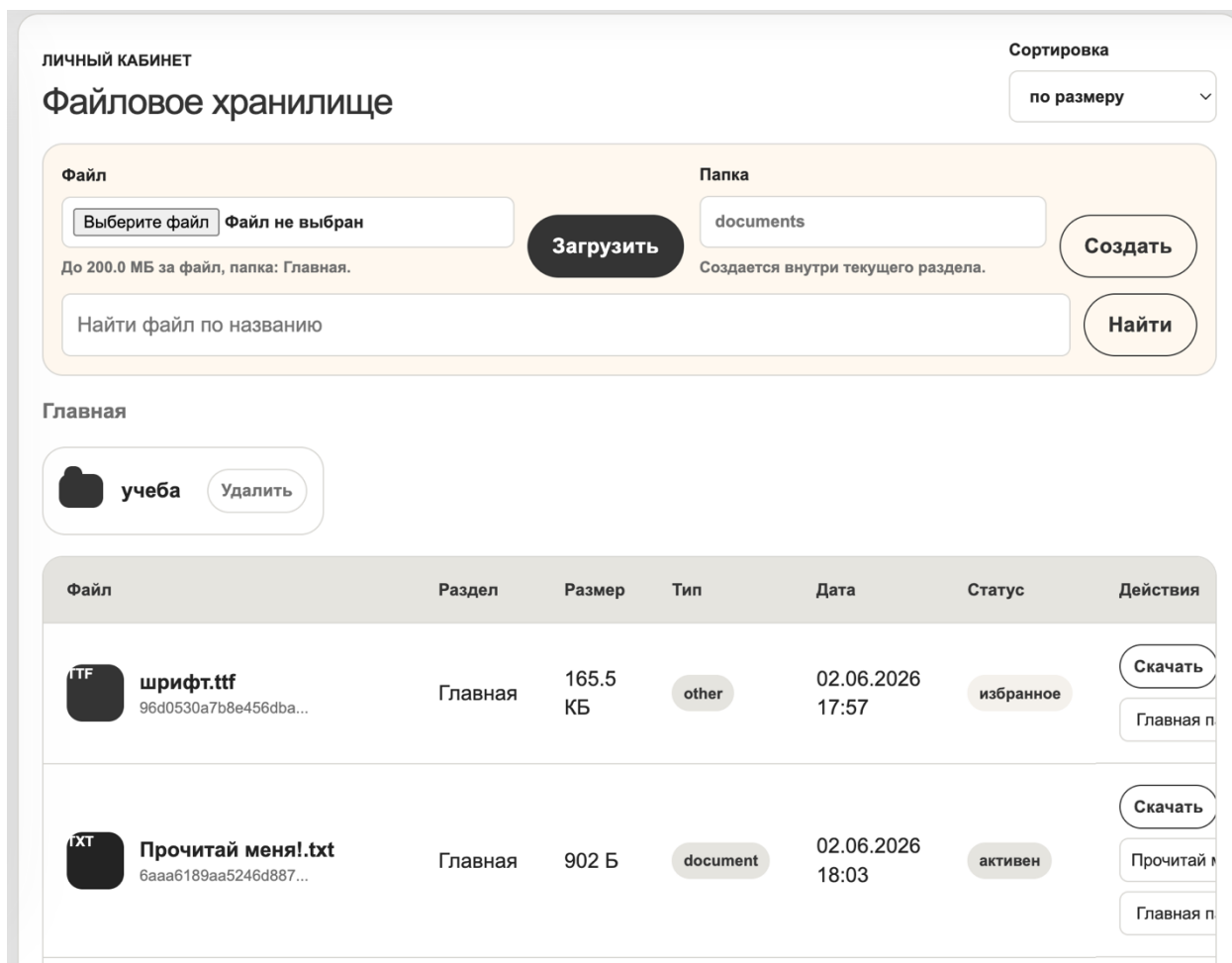


Рисунок 5.11 – Сортировка файлов в личном кабинете.

Если пользователь обладает правами администратора, после входа в систему ему доступна административная панель. В ней отображаются список зарегистрированных пользователей, общая статистика файлового хранилища и журнал действий. Администратор может просматривать сведения о пользователях, блокировать и разблокировать учётные записи, а также удалять пользователей из системы.

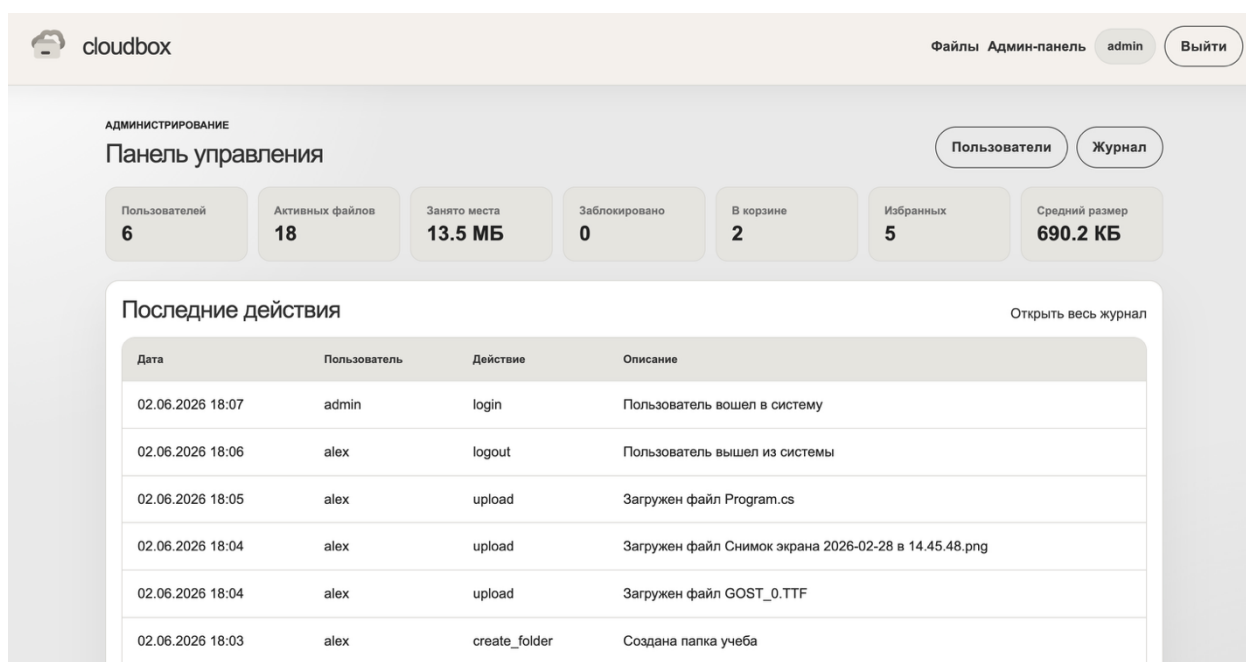


Рисунок 5.12 – Административная панель CloudBox.

Журнал действий предназначен для просмотра основных событий, происходящих в системе. В нём фиксируются операции регистрации, входа, загрузки файлов, удаления, восстановления и административных действий. Наличие журнала позволяет администратору контролировать активность пользователей и анализировать работу приложения.

После завершения работы с приложением пользователь должен выполнить выход из системы. Для этого используется кнопка выхода. После выхода данные пользовательской сессии очищаются, и доступ к личному кабинету становится невозможным до повторной авторизации. Это особенно важно при работе на общем компьютере, так как предотвращает доступ посторонних лиц к личному файловому хранилищу.

Таким образом, веб-приложение CloudBox предоставляет пользователю удобный интерфейс для хранения и управления файлами через браузер. Пользователь может зарегистрироваться, войти в систему, загружать и скачивать файлы, создавать папки, выполнять поиск и сортировку, работать с избранным и корзиной. Администратор дополнительно получает инструменты для управления пользователями, просмотра статистики и контроля действий в системе.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсового проекта было разработано веб-приложение облачного файлового хранилища CloudBox. Программное средство предназначено для хранения пользовательских файлов на сервере и управления ими через браузер. Разработанное приложение позволяет пользователю работать с личным файловым пространством без прямого обращения к файловой системе сервера.

В ходе работы была изучена предметная область облачных файловых хранилищ и рассмотрены существующие решения, такие как Google Drive, Яндекс Диск и Dropbox. На основе проведённого анализа были определены основные функции, необходимые для разрабатываемой системы: регистрация и авторизация пользователей, загрузка файлов, просмотр списка файлов, скачивание, удаление, восстановление, поиск, сортировка и работа с папками.

Серверная часть приложения реализована на языке программирования Python с использованием веб-фреймворка Flask. Приложение построено на основе клиент-серверной архитектуры, при которой пользователь взаимодействует с системой через браузер, а сервер принимает и обрабатывает HTTP-запросы. Для формирования веб-страниц используется шаблонизатор *Jinja2*, позволяющий динамически отображать данные пользователя, список файлов, сообщения системы и элементы управления.

В проекте реализована модульная структура. Отдельные модули отвечают за маршруты приложения, бизнес-логику, работу с JSON-файлами и описание основных сущностей. Такой подход позволил разделить обработку запросов, хранение данных и выполнение операций с файлами. Благодаря этому код приложения стал более понятным, удобным для сопровождения и дальнейшего расширения.

Для хранения данных был использован файловый подход. Метаданные пользователей, файлов и журнала действий сохраняются в JSON-файлах, а сами пользовательские файлы размещаются в файловой системе сервера. Для каждого пользователя создаётся отдельное пространство хранения, что позволяет логически разделить данные разных учётных записей. Такой способ хранения подходит для учебного проекта, так как не требует использования внешней базы данных и наглядно демонстрирует работу с серверными файлами.

В приложении реализованы механизмы авторизации и разграничения прав доступа. Пароли пользователей сохраняются в виде хеша, а доступ к закрытым страницам контролируется с помощью сессий Flask и специальных проверок. Обычный пользователь может работать только со своими файлами,

а администратор получает доступ к панели управления, где может просматривать пользователей, статистику системы и журнал действий.

Особое внимание было уделено функциям работы с файлами. Пользователь может загружать файлы на сервер, скачивать их, переименовывать, перемещать между папками, удалять в корзину, восстанавливать и окончательно удалять. Также реализованы поиск, сортировка и добавление файлов в избранное. Наличие данных функций делает приложение похожим на простой веб-файловый менеджер и обеспечивает удобную работу с хранилищем.

В ходе тестирования была проверена работоспособность основных функций системы. С помощью программы Wireshark был выполнен анализ сетевого взаимодействия между браузером и Flask-сервером. Были зафиксированы HTTP-запросы открытия страницы входа, передачи данных авторизации, открытия личного кабинета, загрузки файла, скачивания файла и обработки несуществующей страницы. Это подтвердило корректную работу клиент-серверного взаимодействия и обработку запросов серверной частью приложения.

Результаты функционального тестирования показали, что приложение корректно выполняет регистрацию и вход пользователя, отображает личный кабинет, загружает и скачивает файлы, выполняет поиск, сортировку, удаление и восстановление объектов. Также была проверена работа административной панели и обработка ошибочных запросов. Все основные функции, заявленные при проектировании, были реализованы и успешно протестированы.

Таким образом, цель курсового проекта была достигнута. Было разработано работоспособное веб-приложение CloudBox, обеспечивающее хранение и управление пользовательскими файлами через браузер. Разработанная система демонстрирует основные принципы построения клиент-серверных веб-приложений, работу с HTTP-запросами, пользовательскими сессиями, файловой системой сервера и JSON-хранилищем данных.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

- [1] Python Documentation [Электронный ресурс]. – Режим доступа: <https://docs.python.org/>. – Дата доступа: 02.06.2026.
- [2] Flask Documentation [Электронный ресурс]. – Режим доступа: <https://flask.palletsprojects.com/>. – Дата доступа: 02.06.2026.
- [3] Jinja Documentation [Электронный ресурс]. – Режим доступа: <https://jinja.palletsprojects.com/>. – Дата доступа: 02.06.2026.
- [4] Werkzeug Documentation. Security Helpers [Электронный ресурс]. – Режим доступа: <https://werkzeug.palletsprojects.com/en/stable/utils/>. – Дата доступа: 02.06.2026.
- [5] MDN Web Docs. Overview of HTTP [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview>. – Дата доступа: 02.06.2026.
- [6] JSON.org. Introducing JSON [Электронный ресурс]. – Режим доступа: <https://www.json.org/>. – Дата доступа: 02.06.2026.
- [7] MDN Web Docs. HTML: HyperText Markup Language [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org/en-US/docs/Web/HTML>. – Дата доступа: 02.06.2026.
- [8] MDN Web Docs. CSS: Cascading Style Sheets [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org/en-US/docs/Web/CSS>. – Дата доступа: 02.06.2026.
- [9] Bootstrap Documentation [Электронный ресурс]. – Режим доступа: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>. – Дата доступа: 02.06.2026.
- [10] Wireshark User's Guide [Электронный ресурс]. – Режим доступа: https://www.wireshark.org/docs/wsug_html_chunked/. – Дата доступа: 02.06.2026.
- [11] Google Drive: Share Files Online with Secure Cloud Storage [Электронный ресурс]. – Режим доступа: <https://workspace.google.com/products/drive/>. – Дата доступа: 02.06.2026.
- [12] Google Drive Help. Upload files & folders to Google Drive [Электронный ресурс]. – Режим доступа: <https://support.google.com/drive/answer/2424368>. – Дата доступа: 02.06.2026.
- [13] Yandex Disk [Электронный ресурс]. – Режим доступа: <https://360.yandex.com/disk/>. – Дата доступа: 02.06.2026.
- [14] Yandex 360 Help. Upload files and folders to Yandex Disk [Электронный ресурс]. – Режим доступа: <https://yandex.com/support/yandex-360/customers/disk/web/en/uploading>. – Дата доступа: 02.06.2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода с комментариями

Содержимое файла app.py:

```
import os

from flask import Flask, flash, g, redirect, render_template, request,
session, url_for

from werkzeug.exceptions import RequestEntityTooLarge

from config import Config
from repositories.user_repository import UserRepository
from routes.admin_routes import admin_bp
from routes.auth_routes import auth_bp
from routes.file_routes import files_bp
from services.auth_service import AuthService
from utils.formatters import format_datetime, format_size

def create_app():
    app = Flask(__name__)
    app.config.from_object(Config)
    Config.init_app()

    user_repository = UserRepository(Config.USERS_FILE)
    auth_service = AuthService(user_repository)
    auth_service.ensure_default_admin()

    @app.before_request
    def load_current_user():
        user_id = session.get("user_id")
        g.current_user = user_repository.get_by_id(user_id) if user_id else
None

    @app.context_processor
```

```

def inject_current_user():
    return {
        "current_user": getattr(g, "current_user", None),
        "upload_max_bytes": Config.UPLOAD_MAX_BYTES,
    }

@app.template_filter("filesize")
def filesize_filter(value):
    return format_size(value)

@app.template_filter("datetime")
def datetime_filter(value):
    return format_datetime(value)

@app.route("/")
def index():
    if getattr(g, "current_user", None):
        return redirect(url_for("files.dashboard"))
    return redirect(url_for("auth.login"))

@app.errorhandler(RequestEntityTooLarge)
def file_too_large(error):
    message = (
        "Размер загружаемого файла превышает ограничение "
        f"{format_size(Config.UPLOAD_MAX_BYTES)}."
    )
    if getattr(g, "current_user", None):
        flash(message, "danger")
        return redirect(url_for("files.dashboard"))

    return (
        render_template(
            "error.html",
            title="Файл слишком большой",
            message=message,
        ),
    )

```



```

        413,
    )

@app.errorhandler(404)
def page_not_found(error):
    return (
        render_template(
            "error.html",
            title="Страница не найдена",
            message="Запрошенная страница не существует.",
        ),
        404,
    )

app.register_blueprint(auth_bp)
app.register_blueprint(files_bp)
app.register_blueprint(admin_bp)

return app

app = create_app()

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5002))
    app.run(debug=True, host="0.0.0.0", port=port)

Содержимое файла routes/decorators.py:

from functools import wraps

from flask import flash, g, redirect, request, url_for

def login_required(view):

```

```

@wraps(view)
def wrapped_view(*args, **kwargs):
    if not getattr(g, "current_user", None):
        flash("Сначала войдите в систему.", "warning")
        return redirect(url_for("auth.login", next=request.path))
    return view(*args, **kwargs)

return wrapped_view

def admin_required(view):
    @wraps(view)
    def wrapped_view(*args, **kwargs):
        current_user = getattr(g, "current_user", None)
        if not current_user:
            flash("Сначала войдите в систему.", "warning")
            return redirect(url_for("auth.login", next=request.path))
        if not current_user.is_admin:
            flash("Доступ разрешен только администратору.", "danger")
            return redirect(url_for("files.dashboard"))
        return view(*args, **kwargs)

    return wrapped_view

```

Содержимое файла routes/auth_routes.py:

```

from urllib.parse import urlsplit

from flask import Blueprint, flash, g, redirect, render_template, request,
session, url_for

from config import Config
from repositories.user_repository import UserRepository
from services.auth_service import AuthService
from services.log_service import LogService

```

```

auth_bp = Blueprint("auth", __name__)

user_repository = UserRepository(Config.USERS_FILE)
auth_service = AuthService(user_repository)
log_service = LogService()

def _safe_next_url(next_url):
    if not next_url:
        return None

    parsed = urlsplit(next_url)
    if parsed.scheme or parsed.netloc:
        return None

    if not next_url.startswith("/") or next_url.startswith("//"):
        return None

    return next_url

@auth_bp.route("/register", methods=["GET", "POST"])
def register():
    if getattr(g, "current_user", None):
        return redirect(url_for("files.dashboard"))

    if request.method == "POST":
        success, message, user = auth_service.register_user(
            request.form.get("username"),
            request.form.get("email"),
            request.form.get("password"),
            request.form.get("password_repeat"),
        )

        flash(message, "success" if success else "danger")

        if success:
            log_service.add(user, "register", "Зарегистрирован новый пользователь")

            return redirect(url_for("auth.login"))

```

```

return render_template("register.html", title="Регистрация")

@auth_bp.route("/login", methods=["GET", "POST"])
def login():
    if getattr(g, "current_user", None):
        return redirect(url_for("files.dashboard"))

    if request.method == "POST":
        success, message, user = auth_service.login_user(
            request.form.get("email"),
            request.form.get("password"),
        )
        flash(message, "success" if success else "danger")
        if success:
            session.clear()
            session["user_id"] = user.id
            log_service.add(user, "login", "Пользователь вошел в систему")
            next_url = _safe_next_url(request.args.get("next"))
            return redirect(next_url or url_for("files.dashboard"))

    return render_template("login.html", title="Вход")

@auth_bp.route("/logout")
def logout():
    current_user = getattr(g, "current_user", None)
    if current_user:
        log_service.add(current_user, "logout", "Пользователь вышел из системы")
        session.clear()
        flash("Вы вышли из системы.", "info")
        return redirect(url_for("auth.login"))

```

Содержимое файла routes/file_routes.py:

```

from flask import (
    Blueprint,
    flash,
    g,
    redirect,
    render_template,
    request,
    send_file,
    url_for,
)

from config import Config
from services.file_service import FileService
from routes.decorators import login_required
from utils.formatters import format_size

files_bp = Blueprint("files", __name__)
file_service = FileService()

def _upload_request_too_large():
    content_length = request.content_length
    if not content_length:
        return False
    return content_length > Config.UPLOAD_MAX_BYTES +
Config.UPLOAD_FORM_OVERHEAD_BYTES

def _redirect_to_dashboard(folder="", query=None, view="folder",
sort_by="name"):
    params = {}
    if folder and view == "folder":
        params["folder"] = folder
    if view != "folder":
        params["view"] = view
    if sort_by != "name":

```

```

        params["sort"] = sort_by
    if query:
        params["q"] = query
    return redirect(url_for("files.dashboard", **params))

@files_bp.route("/dashboard")
@login_required
def dashboard():
    current_user = g.current_user
    folder = request.args.get("folder", "")
    query = request.args.get("q", "").strip()
    view = request.args.get("view", "folder")
    sort_by = request.args.get("sort", "name")
    if view not in {"folder", "favorites", "trash"}:
        view = "folder"

    try:
        normalized_folder = file_service.normalize_folder(folder)
        if query:
            files = file_service.search_files(current_user.id, query,
view=view, sort_by=sort_by)
            folders = []
        else:
            files = file_service.get_files(
                current_user.id,
                normalized_folder,
                view=view,
                sort_by=sort_by,
            )
            folders = (
                file_service.list_folders(current_user.id, normalized_folder)
                if view == "folder"
                else []
            )

```

```

        return render_template(
            "dashboard.html",
            title="Личный кабинет",
            files=files,
            folders=folders,
            all_folders=file_service.list_all_folders(current_user.id),
            breadcrumbs=file_service.get_breadcrumbs(normalized_folder),
            current_folder=normalized_folder,
            query=query,
            view=view,
            sort_by=sort_by,
            storage_info=file_service.get_storage_info(current_user.id),
            summary=file_service.get_dashboard_summary(current_user.id),
        )
    except ValueError as error:
        flash(str(error), "danger")
        return _redirect_to_dashboard()

@files_bp.route("/upload", methods=["POST"])
@login_required
def upload():
    if _upload_request_too_large():
        flash(f"Размер файла превышает лимит {format_size(Config.UPLOAD_MAX_BYTES)}.", "danger")
        return _redirect_to_dashboard()

    folder = request.form.get("folder", "")
    sort_by = request.form.get("sort", "name")
    redirect_folder = ""
    try:
        redirect_folder = file_service.normalize_folder(folder)
        success, message, _file_item = file_service.upload_file(
            g.current_user,
            request.files.get("file"),
            redirect_folder,

```

```

    )

    flash(message, "success" if success else "danger")
except ValueError as error:
    flash(str(error), "danger")

return _redirect_to_dashboard(redirect_folder, sort_by=sort_by)

@files_bp.route("/download/<int:file_id>")
@login_required
def download(file_id):
    file_item = file_service.get_file_info(g.current_user.id, file_id)
    if not file_item:
        flash("Файл не найден.", "danger")
        return _redirect_to_dashboard()

    file_path = file_service.get_file_path(file_item)
    if not file_path.exists():
        flash("Файл отсутствует на сервере.", "danger")
        return _redirect_to_dashboard(file_item.folder)

    return send_file(file_path, as_attachment=True,
download_name=file_item.original_name)

@files_bp.route("/delete/<int:file_id>", methods=["POST"])
@login_required
def delete(file_id):
    folder = request.form.get("folder", "")
    view = request.form.get("view", "folder")
    sort_by = request.form.get("sort", "name")
    success, message = file_service.delete_file(g.current_user, file_id)
    flash(message, "success" if success else "danger")
    return _redirect_to_dashboard(folder, view=view, sort_by=sort_by)

```



```

@files_bp.route("/restore/<int:file_id>", methods=["POST"])
@login_required
def restore(file_id):
    sort_by = request.form.get("sort", "name")
    success, message = file_service.restore_file(g.current_user, file_id)
    flash(message, "success" if success else "danger")
    return _redirect_to_dashboard(view="trash", sort_by=sort_by)


@files_bp.route("/destroy/<int:file_id>", methods=["POST"])
@login_required
def destroy(file_id):
    sort_by = request.form.get("sort", "name")
    success, message = file_service.permanent_delete_file(g.current_user,
file_id)
    flash(message, "success" if success else "danger")
    return _redirect_to_dashboard(view="trash", sort_by=sort_by)


@files_bp.route("/trash/empty", methods=["POST"])
@login_required
def empty_trash():
    sort_by = request.form.get("sort", "name")
    success, message = file_service.empty_trash(g.current_user)
    flash(message, "success" if success else "info")
    return _redirect_to_dashboard(view="trash", sort_by=sort_by)


@files_bp.route("/favorite/<int:file_id>", methods=["POST"])
@login_required
def favorite(file_id):
    folder = request.form.get("folder", "")
    view = request.form.get("view", "folder")
    sort_by = request.form.get("sort", "name")
    success, message = file_service.toggle_favorite(g.current_user, file_id)
    flash(message, "success" if success else "danger")

```

```

        return _redirect_to_dashboard(folder, view=view, sort_by=sort_by)

@files_bp.route("/rename/<int:file_id>", methods=["POST"])
@login_required
def rename(file_id):
    folder = request.form.get("folder", "")
    view = request.form.get("view", "folder")
    sort_by = request.form.get("sort", "name")
    new_name = request.form.get("new_name", "")
    try:
        success, message, _file_item =
file_service.rename_file(g.current_user, file_id, new_name)
        flash(message, "success" if success else "danger")
    except ValueError as error:
        flash(str(error), "danger")
    return _redirect_to_dashboard(folder, view=view, sort_by=sort_by)

@files_bp.route("/move/<int:file_id>", methods=["POST"])
@login_required
def move(file_id):
    folder = request.form.get("folder", "")
    view = request.form.get("view", "folder")
    sort_by = request.form.get("sort", "name")
    target_folder = request.form.get("target_folder", "")
    try:
        success, message = file_service.move_file(g.current_user, file_id,
target_folder)
        flash(message, "success" if success else "danger")
    except ValueError as error:
        flash(str(error), "danger")
    return _redirect_to_dashboard(folder, view=view, sort_by=sort_by)

@files_bp.route("/folder/create", methods=["POST"])
@login_required

```

```

def create_folder():
    current_folder = request.form.get("folder", "")
    sort_by = request.form.get("sort", "name")
    folder_name = request.form.get("folder_name", "")
    redirect_folder = ""

    try:
        redirect_folder = file_service.normalize_folder(current_folder)
        success, message, _new_folder = file_service.create_folder(
            g.current_user,
            redirect_folder,
            folder_name,
        )
        flash(message, "success" if success else "danger")
    except ValueError as error:
        flash(str(error), "danger")

    return _redirect_to_dashboard(redirect_folder, sort_by=sort_by)


@files_bp.route("/folder/delete", methods=["POST"])
@login_required
def delete_folder():
    current_folder = request.form.get("current_folder", "")
    target_folder = request.form.get("target_folder", "")
    sort_by = request.form.get("sort", "name")
    redirect_folder = ""

    try:
        redirect_folder = file_service.normalize_folder(current_folder)
        success, message = file_service.delete_folder(g.current_user,
            target_folder)
        flash(message, "success" if success else "danger")
    except ValueError as error:
        flash(str(error), "danger")

    return _redirect_to_dashboard(redirect_folder, sort_by=sort_by)

```

```

@files_bp.route("/search")
@login_required
def search():
    query = request.args.get("q", "").strip()
    folder = request.args.get("folder", "")
    view = request.args.get("view", "folder")
    sort_by = request.args.get("sort", "name")
    return _redirect_to_dashboard(folder=folder, query=query, view=view,
sort_by=sort_by)

```

```

@files_bp.route("/file/<int:file_id>")
@login_required
def file_info(file_id):
    file_item = file_service.get_file_info(g.current_user.id, file_id,
include_deleted=True)
    if not file_item:
        flash("Файл не найден.", "danger")
        return _redirect_to_dashboard()

    file_path = file_service.get_file_path(file_item)
    return render_template(
        "file_info.html",
        title="Информация о файле",
        file_item=file_item,
        file_exists=file_path.exists(),
    )

```

Содержимое файла routes/admin_routes.py:

```

from flask import Blueprint, flash, g, redirect, render_template, request,
url_for

from routes.decorators import admin_required
from services.admin_service import AdminService

```

```

admin_bp = Blueprint("admin", __name__, url_prefix="/admin")
admin_service = AdminService()

@admin_bp.route("")
@admin_required
def dashboard():
    return render_template(
        "admin.html",
        title="Админ-панель",
        statistics=admin_service.get_statistics(),
        recent_logs=admin_service.get_logs()[:8],
    )

@admin_bp.route("/users")
@admin_required
def users():
    return render_template(
        "admin_users.html",
        title="Пользователи",
        users=admin_service.get_users(),
    )

@admin_bp.route("/users/delete", methods=["POST"])
@admin_required
def delete_user():
    user_id = request.form.get("user_id")
    success, message = admin_service.delete_user(g.current_user, user_id)
    flash(message, "success" if success else "danger")
    return redirect(url_for("admin.users"))

@admin_bp.route("/users/block", methods=["POST"])
@admin_required

```

```

def block_user():
    user_id = request.form.get("user_id")
    is_blocked = request.form.get("is_blocked") == "1"
    success, message = admin_service.set_user_blocked(g.current_user,
user_id, is_blocked)
    flash(message, "success" if success else "danger")
    return redirect(url_for("admin.users"))


@admin_bp.route("/statistics")
@admin_required
def statistics():
    return redirect(url_for("admin.dashboard"))


@admin_bp.route("/logs")
@admin_required
def logs():
    return render_template(
        "admin_logs.html",
        title="Журнал действий",
        logs=admin_service.get_logs(),
    )
}

```

ПРИЛОЖЕНИЕ Б
(обязательное)
Схема алгоритма работы системы